

COMPOSE: GENERATIVE STOCHASTIC REWRITING FOR TRANS-DIMENSIONAL MOLECULAR DESIGN AND CONTROL

Anonymous authors

Paper under double-blind review

ABSTRACT

Controllable molecular design requires both a representation of how molecules can change and a mechanism for steering those changes toward a desired outcome. Yet molecular generation and optimization are typically built around fixed endpoint distributions or task objectives, rather than a reusable representation of molecular transitions that can be learned once and controlled across changing design goals. We introduce COMPOSE, a generative framework for molecular design that represents generation as a stochastic sequence of executable edits between complete molecular graphs. We learn a goal-independent reference law $R_\theta(y \mid x)$ that assigns probability across the legal, trans-dimensional successors of each molecule, including atom insertion/deletion, bond changes, and ring operations. Iterating this law defines a stochastic molecular reference process. This reference process is learned once and held fixed, while a separate finite-horizon controller supplies task-specific purpose by steering the same learned transition process toward different objectives. With the same frozen reference process across objectives, COMPOSE is competitive with specialized methods on established single-objective editing and multi-objective optimization benchmarks under fixed oracle budgets. Beyond these benchmark settings, we show that the same formulation enables richer trajectory-level control. By representing every intermediate as a complete molecule with an explicit set of legal successors, COMPOSE improves molecular optimization by reasoning over the futures each edit makes reachable and by reusing reached states to adaptively target underexplored regions of the Pareto front. The same process supports mid-trajectory retargeting and constraints that hold throughout generation. COMPOSE learns one reusable molecular world and then lets us control, redirect, constrain, and reuse molecular trajectories in ways that ordinary endpoint-generation frameworks do not naturally expose.

1 INTRODUCTION

Generative modeling has become a central approach to molecular design, supporting both de novo generation and the optimization of existing molecules across increasingly complex chemical spaces (Gómez-Bombarelli et al., 2018; Jin et al., 2018; 2019; Vignac et al., 2023; Campbell et al., 2024; Lee et al., 2025). Recent graph-based diffusion, flow, and discrete generative models can learn expressive molecular distributions and increasingly support conditional generation toward desired structural and functional properties (Vignac et al., 2023; Campbell et al., 2024; Lee et al., 2025). These advances have made molecular generation substantially more flexible, but the learned object is still typically an endpoint distribution or a generator specialized toward a particular task, rather than a reusable representation of transitions between molecular states. For molecular design, this suggests value in modeling the path between structures, since each transformation changes both the next molecule and the set of molecular states that remain reachable afterward.

Molecular editing makes this path dependence explicit, because each edit changes the transformations available from the resulting molecule. Existing editing and lead-optimization methods already exploit sequential molecular modifications through graph-to-graph generation, reinforcement learning, fragment-based search, evolutionary optimization, and MCMC (Jin et al., 2019; Zhou et al., 2019; Jensen, 2019; Xie et al., 2021; Fu et al., 2021). Yet the consequence of an edit extends beyond the molecule it immediately produces. An atom insertion, bond modification, or ring operation can open or eliminate entire downstream branches of the design space. The edit with the largest

immediate property gain may therefore be a poor choice under a finite design budget, while a locally unremarkable edit may create access to substantially better molecules several steps later. This distinction is consequential in lead optimization, where gains in one property can create liabilities in others, such as improving potency while compromising solubility or selectivity, and where finite evaluation budgets make it valuable to preserve alternative molecular profiles rather than repeatedly optimize the same trade-off. New measurements may also introduce requirements that were absent from the original objective, making already-reached molecules useful starting points for a revised design goal. Likewise, requirements such as preserving an important molecular feature or remaining within a predictor’s trusted chemical regime may need to hold throughout optimization rather than only at the returned molecule. Molecular editing may therefore benefit from representing not only the value of individual structures, but also which molecular futures remain reachable from each state.

Despite this progress, existing approaches capture only parts of a transition-centered formulation. Learned generators can increasingly be guided toward desired properties, but their generative trajectories are often defined through corrupted, masked, or latent states rather than reusable transitions between complete molecules (Vignac et al., 2023; Campbell et al., 2024; Lee et al., 2025). Molecular editing and search methods instead operate directly on complete molecules, but their proposal mechanisms are typically prescribed by the optimizer and coupled to task-specific scoring (Jensen, 2019; Zhou et al., 2019; Xie et al., 2021). Recent work further shows that pretrained discrete generators can be redirected toward multiple objectives and constraints through inference-time guidance and control (Chen et al., 2025; Anonymous, 2026). Together, these advances motivate a complementary abstraction, a goal-independent generative process over molecular transitions that can be learned once and controlled as design goals change.

We introduce **COMPOSE (Controllable Molecular Process Over Stochastic Edits)**, a generative framework that learns a reusable molecular transition process and controls it across design objectives (Figure 1). COMPOSE represents generation as a stochastic sequence of executable edits between complete molecules. At each molecular state, an exact executor defines what is possible by enumerating the state-dependent set of legal, trans-dimensional molecular successors. We then learn a goal-independent reference law R_θ over this same successor set from executable molecular trajectories, capturing which molecular continuations are plausible without conditioning on a downstream design goal. A separate finite-horizon controller determines what is purposeful by reweighting these molecular continuations according to the value of the futures they make reachable. Within this formulation, future-aware edit selection, exploration across competing property trade-offs, retargeting from already-reached molecules, and statewise constraints are different interventions on the same learned molecular process rather than separate task-specific generators. Because generation and control operate on the same explicit molecular support, the transition process can be learned once and redirected across design objectives without retraining the underlying generator. Our main contributions are fourfold.

1. We introduce COMPOSE, a learned stochastic process over executable, trans-dimensional molecular rewrites between complete molecular graphs, with generation and control defined on canonical molecular successors.
2. We formulate molecular design as finite-horizon control of this learned process and derive a budget-indexed Doob h -transform that introduces no transitions outside the reference support and realizes exact terminal reweighting under exact future values.
3. We develop a goal-conditioned future-value controller that steers the same frozen reference process across single- and multi-objective molecular design tasks without retraining the underlying generator.
4. We demonstrate COMPOSE on established molecular editing and multi-objective optimization benchmarks, and show that future-aware control, multi-objective Pareto exploration, mid-trajectory retargeting, and pathwise constraints exploit capabilities of the learned molecular transition process beyond endpoint scoring.

2 RELATED WORK

Molecular generation spans latent-variable, graph-to-graph, reinforcement-learning, diffusion, and flow-based approaches, while molecular optimization has also been formulated directly through

108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161

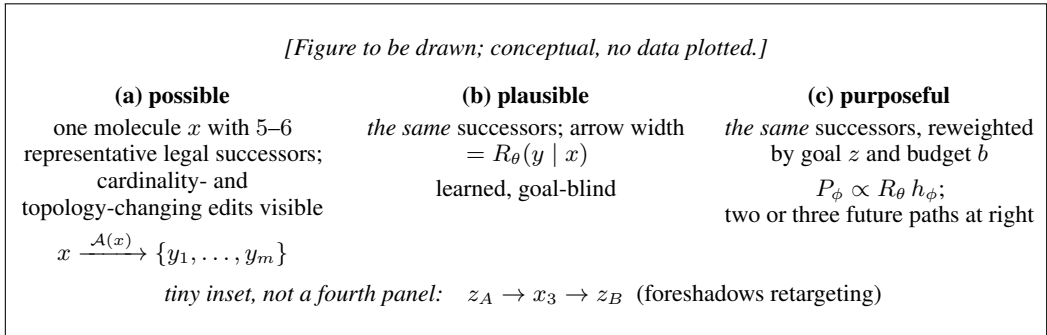


Figure 1: Possible, plausible, purposeful. An exact executor defines the legal molecular successors of a state, the goal-independent reference process R_θ learns probability over those transitions, and a finite-horizon controller reweights the same process toward the current design objective. All three panels act on one successor set. When the objective changes, only the controller changes.

graph edits, fragment proposals, evolutionary operators, and MCMC transitions (Gómez-Bombarelli et al., 2018; Jin et al., 2018; 2019; Zhou et al., 2019; Jensen, 2019; Xie et al., 2021; Fu et al., 2021; Vignac et al., 2023; Campbell et al., 2024). Recent work has begun to relax the fixed-size assumption of graph generation. GrIDDD introduces node insertion and deletion into discrete graph diffusion, allowing molecular graphs to grow and shrink during denoising (Ninniri et al., 2025). COMPOSE instead learns a goal-independent stochastic law over an explicit, state-dependent molecular rewrite space, making the transition process itself the object of subsequent control.

A related line of work treats generation and adaptation directly through stochastic processes. Generator matching provides a general framework for learning Markov generators from tractable conditional processes, while Edit Flows develops insertion, deletion, and substitution dynamics for variable-length discrete generation (Holderrieth et al., 2025; Havasi et al., 2025). MadSBM is a close mathematical relative, formulating peptide generation as a controlled Markov process on an amino-acid edit graph, although its learned control constructs the generative transport rather than adapting an already learned molecular process (Goel & Chatterjee, 2026). Value Matching steers a frozen flow model through a learned value function, and pCoMole studies Pareto-constrained molecular editing with discrete flows (Jensen et al., 2026; Anonymous, 2026). Future-aware generative control is therefore not unique to COMPOSE. Value-guided flow adaptation and pCoMole likewise use learned or rollout-estimated future value. Our distinction is the substrate on which that control operates. It is a learned canonical transition law over executable molecular graphs, which is what allows realized molecular states to be reused, retargeted, and constrained.

Multi-objective molecular design has been addressed through surrogate-based optimization, evolutionary search, and inference-time guidance of generative models. InversionGNN combines multi-property prediction with gradient-based Pareto search in discrete molecular space (Niu et al., 2025), while PepTune and AReURDi use discrete generative models with multi-objective tree search or Pareto-oriented refinement (Tang et al., 2025; Chen et al., 2025). Routing by Reaching similarly composes pretrained GFlowNets for multi-objective generation at inference time (Yoon et al., 2026). COMPOSE instead places multi-objective control on a reusable molecular transition process, so that future reachability and already-reached molecular states can participate directly in Pareto exploration.

3 PRELIMINARIES

3.1 MARKOV CHAINS

Let $\mathcal{X}$ be a discrete state space and $P_0, \dots, P_{K-1}$ a sequence of transition kernels defining a possibly time-inhomogeneous Markov chain over K steps. Starting from $X_0 = x_0$, the probability of a trajectory $x_{1:K}$ is

$$P(X_{1:K} = x_{1:K} \mid X_0 = x_0) = \prod_{k=0}^{K-1} P_k(x_{k+1} \mid x_k). \quad (1)$$

Thus each transition depends only on the current state and the step of the process, while the sequence of kernels determines a distribution over complete trajectories. In COMPOSE, fixed-budget molecular editing takes this form, with each state a complete molecular graph and each committed transition one molecular edit.

3.2 FINITE-HORIZON DOOB CONTROL

We also recall finite-horizon control of a reference Markov chain. Let $g_z(x) \geq 0$ assign terminal desirability under goal z . Defining the backward value $h_K(x, z) = g_z(x)$ and

$$h_k(x, z) = \sum_y P_k(y | x) h_{k+1}(y, z), \quad P_k^z(y | x) = P_k(y | x) \frac{h_{k+1}(y, z)}{h_k(x, z)}, \quad (2)$$

gives the finite-horizon Doob h -transform wherever $h_k(x, z) > 0$. Writing P^z for the law of the chain run under these controlled kernels, the resulting terminal distribution is

$$P^z(X_K = y | X_0 = x_0) = \frac{P(X_K = y | X_0 = x_0) g_z(y)}{\mathbb{E}_P[g_z(X_K) | X_0 = x_0]}. \quad (3)$$

The transform therefore reweights only transitions already present in the reference chain and introduces no new support. With exact backward values, it realizes the reference terminal distribution tilted by g_z . In COMPOSE, this construction provides the control law for molecular editing, with an amortized future-value model replacing exact backward computation in large molecular spaces.

COMPOSE instantiates these generative and control primitives on a state-dependent, trans-dimensional molecular rewrite space.

4 METHODS

We develop COMPOSE as a reusable stochastic process over molecular edits and a finite-horizon controller over that process. COMPOSE realizes generative stochastic rewriting by repeatedly applying executable molecular edits under a learned, goal-independent transition law. An exact rewrite system first determines which transitions between complete molecular graphs are executable. We then learn a goal-independent reference law over this state-dependent support and push it forward to a canonical molecular successor kernel R_θ . Finally, we control the frozen successor process through a budget-conditioned h -transform, using a learned future-value model h_ϕ to approximate the required backward values at molecular scale.

4.1 EXECUTABLE MOLECULAR TRANSITIONS

COMPOSE represents every generative state as a complete molecular graph. Let $n_{\max}$ denote the maximum number of heavy atoms admitted by the declared molecular state space. For $n \in \{1, \dots, n_{\max}\}$, let $\mathcal{X}_n$ be the set of connected molecular graphs with n heavy atoms whose node and edge attributes lie in the declared vocabularies of elements, bond orders, formal charges, and implicit hydrogens, and in which every atom satisfies its declared valence constraints. The molecular state space is

$$\mathcal{X} = \bigsqcup_{n=1}^{n_{\max}} \mathcal{X}_n. \quad (4)$$

Thus each state $x \in \mathcal{X}$ is a complete molecule rather than a masked, corrupted, or partially specified representation.

Molecular transitions are generated by typed graph rewrites. An edit mark a specifies a rewrite type together with its molecular match and typed payload, and $T(x, a)$ denotes the graph obtained by applying that rewrite to x . The full atom-, bond-, and ring-level edit language is specified in Appendix A.2. From a state x , the executor exposes only the finite, state-dependent set

$$\mathcal{A}(x) = \{a : a \text{ matches } x \text{ and } T(x, a) \text{ passes all executor guards}\}. \quad (5)$$

Execution follows a propose–validate–commit rule, and a successor is admitted only if its attributes remain in the declared vocabularies, affected valences are satisfied, the graph remains connected,

and its heavy-atom count lies in $\{1, \dots, n_{\max}\}$. An edit that fails any guard contributes no transition to $\mathcal{A}(x)$. Inadmissible molecular states are therefore absent from the stochastic support rather than discouraged through a learned penalty.

The decomposition of $\mathcal{X}$ by heavy-atom count makes the process trans-dimensional. Each admitted edit changes heavy-atom count by at most one, yielding *birth*, *death*, or *same-cardinality* transitions between $\mathcal{X}_n$ and $\mathcal{X}_{n+1}$, $\mathcal{X}_{n-1}$, or $\mathcal{X}_n$, respectively. Birth and death edits create or remove typed graph entities that participate in valence, connectivity, and ring perception rather than merely activating or deactivating padded coordinates. A trajectory may therefore grow, shrink, and restructure a molecular graph within a common state space.

Proposition 1 (Closure of executable support). *For any $x \in \mathcal{X}$ and $a \in \mathcal{A}(x)$, the executor guards imply*

$$T(x, a) \in \mathcal{X}.$$

Consequently, every state reached by a stochastic process supported on $\{\mathcal{A}(x)\}_{x \in \mathcal{X}}$ remains in the declared molecular state space. The proof is given in Appendix B.1.

The legal edit sets $\mathcal{A}(x)$ therefore determine which molecular transitions are possible, but not how probability should be distributed among them. We next learn a goal-independent stochastic law over this executable support.

4.2 LEARNED MOLECULAR REFERENCE PROCESS

Having defined which molecular rewrites are executable, we next learn how probability is distributed over them. For every state x with $\mathcal{A}(x) \neq \emptyset$, the reference model defines a normalized conditional law over legal edit marks,

$$p_\theta(a \mid x, t), \quad \sum_{a \in \mathcal{A}(x)} p_\theta(a \mid x, t) = 1, \quad (6)$$

where t is a progress variable. Writing an edit as $a = (k, o, \eta)$, with family k , matched molecular operands o , and typed payload η , we parameterize

$$p_\theta(a \mid x, t) = p_\theta(k \mid x, t) p_\theta(o, \eta \mid k, x, t). \quad (7)$$

A shared molecular encoder represents the current graph, while family-specific scorers normalize only over candidates admitted by the executor. The reference model receives no downstream objective, remaining budget, or task-specific constraint, and the original source molecule is not supplied as an additional conditioning variable. The source determines the initial state, and task information enters later through support restrictions and control. Thus the learned reference law represents molecular *plausibility* independently of molecular *purpose*.

Supervision is obtained by compiling molecular endpoint pairs into executable edit programs. Every stored teacher transition is replayed through the same executor used at inference, so the model is trained only on transitions available to the deployed process. The compiler may use both endpoints to construct an edit program, but neither the target endpoint nor the program is available to the reference model at inference. Because multiple legal marks can execute to the same molecular graph, the learning objective is defined on the resulting molecular successor rather than on a particular syntactic realization. Let $R_{\theta,t}(y \mid x)$ denote the induced law over distinct committed molecular successors, formalized in Section 4.3. For productive teacher transitions $x_i \rightarrow y_i^*$, we minimize

$$\mathcal{L}_{\text{ref}}(\theta) = -\frac{\sum_i w_i \log R_{\theta,t_i}(y_i^* \mid x_i)}{\sum_i w_i}, \quad (8)$$

where w_i is the registered weight under the declared editing measure. Terminal rows do not enter this objective. Equation (8) therefore learns which committed molecular successor follows a state under that measure. It does not calibrate continuous-time holding rates.

This objective is related to generator matching but is not the generator-matching population objective. Writing the productive rate as $Q^+(y \mid x, t) = \Lambda(x, t)P(y \mid x, t)$, the corresponding generator-matching divergence separates pointwise into a productive-hazard term and a conditional-successor term proportional to $\Lambda^* \text{KL}(P^* \parallel R_\theta)$. COMPOSE optimizes the successor component only. The

hazard term is not optimized, and Equation (8) is averaged under the declared family-weighted editing measure rather than the state-time marginal of the conditional-process mixture (Holderrieth et al., 2025). Appendix B.3 gives the full decomposition and the precise measure-level distinction.

The learned mark law is not yet the molecular kernel used for control, because distinct executable marks can yield the same molecular state. We therefore next push p_θ through the executor and aggregate probability by canonical molecular outcome.

4.3 CANONICAL MOLECULAR SUCCESSORS

We now push the learned edit law through the executor to obtain a stochastic process over distinct molecular states. Multiple legal marks can produce the same molecular graph because of graph symmetries, equivalent matches, or alternative internal rewrite descriptions. For a canonical successor y , define its successor fiber

$$\mathcal{G}_y(x) = \{a \in \mathcal{A}(x) : T(x, a) \simeq y\}, \quad (9)$$

where $\simeq$ denotes equivalence under the registered molecular canonicalization. The induced probability of y is

$$\bar{p}_{\theta,t}(y | x) = \sum_{a \in \mathcal{G}_y(x)} p_\theta(a | x, t). \quad (10)$$

Thus probability assigned to a molecular transition is the aggregate mass of every executable mark that realizes it. For example, deleting any of the three symmetry-equivalent terminal methyl carbons in 2-methylpropane, CC(C)C, produces propane, and canonicalization exposes propane once with the combined probability of those marks rather than overweighting it because of representational multiplicity.

Fixed-budget editing counts committed changes of molecular state rather than internal mark draws. Let $\mathcal{S}(x)$ denote the distinct non-self canonical successors admitted by this convention. Conditioning the pushforward distribution on a committed state change defines the molecular reference kernel

$$R_{\theta,t}(y | x) = \frac{\mathbf{1}\{y \in \mathcal{S}(x)\} \bar{p}_{\theta,t}(y | x)}{\sum_{y' \in \mathcal{S}(x)} \bar{p}_{\theta,t}(y' | x)}. \quad (11)$$

If $\mathcal{S}(x) = \emptyset$, x is terminal for the committed molecular process. For $b > 0$, such a state has $h_b(x, z) = 0$ under the backward recursion of Section 4.4. Trajectories terminating before the declared horizon therefore carry zero mass in the exact- K -step controlled terminal law. Otherwise $R_{\theta,t}$ is a normalized law over distinct molecular successors, and iterating it defines the goal-independent reference process optimized by Equation (8). We suppress t and write $R_\theta(y | x)$ when the registered progress convention is fixed. Canonical self-events and other non-committed outcomes are detailed in Appendix B.2.

Proposition 2 (Canonical normalization and refinement invariance). *For every nonterminal state x , $\sum_{y \in \mathcal{S}(x)} R_{\theta,t}(y | x) = 1$. Moreover, any refinement or coarsening of the internal mark representation that preserves the aggregate probability mass of every canonical successor fiber leaves $R_{\theta,t}$ unchanged. Consequently, state-level controllers that reweight $R_{\theta,t}$ using quantities defined on canonical molecular successors are invariant to such representational refinements.*

This invariance relies on performing control after canonicalization, since reweighting or truncating individual marks beforehand can reintroduce dependence on arbitrary edit multiplicities. With the goal-independent reference process now defined on complete molecular states, we next control this frozen kernel toward finite-horizon design objectives.

4.4 FINITE-HORIZON MOLECULAR CONTROL

Having defined the goal-independent molecular reference process R_θ , we now introduce task-specific control without modifying that process. Let z denote a design objective and $g_z : \mathcal{X} \rightarrow \mathbb{R}_{\geq 0}$ its terminal desirability. For a remaining budget of b molecular transitions, define the backward values

$$h_0(x, z) = g_z(x), \quad h_b(x, z) = \sum_{y \in \mathcal{S}(x)} R_\theta(y | x) h_{b-1}(y, z) = \mathbb{E}_{R_\theta}[g_z(X_b) | X_0 = x]. \quad (12)$$

Thus $h_b(x, z)$ evaluates a molecule by the desirability reachable from it after b further transitions under the reference process. Unlike a score of the current molecule, it depends on the molecular futures that remain accessible through R_θ .

For $h_b(x, z) > 0$, the finite-horizon h -transform defines

$$R_\theta^{z,b}(y | x) = R_\theta(y | x) \frac{h_{b-1}(y, z)}{h_b(x, z)}. \quad (13)$$

The controlled kernel therefore reweights the same canonical molecular successors according to their remaining future value, so a transition may be preferred despite little immediate improvement if it preserves access to desirable states later in the trajectory. If $h_b(x, z) = 0$, no state with positive terminal desirability is reachable within the remaining budget under the reference process, and we leave the controlled row undefined rather than introduce probability outside its support.

Theorem 3 (Finite-horizon molecular control). *Let R_θ define the reference path measure over K committed transitions from x_0 , with trajectories terminating before K assigned zero continuation mass, let g_z be nonnegative and finite-valued, and suppose $0 < h_K(x_0, z) < \infty$. Applying $R_\theta^{z,K-j}$ at step $j = 0, \dots, K - 1$ defines normalized controlled kernels whose support is contained in that of R_θ , and whose terminal distribution satisfies*

$$\mathbb{P}^z(X_K = y | X_0 = x_0) = \mathbb{P}_{R_\theta}(X_K = y | X_0 = x_0) \frac{g_z(y)}{h_K(x_0, z)}. \quad (14)$$

Thus exact backward values realize the g_z -tilted terminal law without introducing transitions outside the executable reference support. The proof is given in Appendix B.1.

The guarantee is horizon-specific. Before the terminal step, the controlled marginal is determined by the remaining backward value rather than directly by g_z , so early stopping does not in general realize the terminal tilt. Because every state of R_θ is a complete molecule and every transition an executable rewrite, $h_b(x, z)$ has a direct interpretation as future desirability from a realized molecular intermediate through its remaining executable continuations. Exact backward evaluation is tractable only when the relevant state graph can be enumerated. Section 4.5 therefore replaces h_b with a learned goal- and budget-conditioned approximation $h_\phi(x, z, b)$, retaining the same finite-horizon control interface at molecular scale.

4.5 LEARNING GOAL- AND BUDGET-CONDITIONED FUTURE VALUE

Exact evaluation of the backward values in Section 4.4 requires propagating terminal desirability over the molecular states reachable within the remaining edit budget, which is infeasible in realistic molecular spaces. We therefore learn an amortized approximation h_ϕ while keeping the molecular reference process R_θ fixed. For objective z and b remaining transitions,

$$h_\phi(x, z, b) \approx h_b(x, z) = \mathbb{E}_{R_\theta}[g_z(X_b) | X_0 = x]. \quad (15)$$

The remaining budget is part of the prediction problem, because the same molecule can have different future value under the same objective depending on how many executable transitions remain.

We construct supervision directly from the frozen reference process. From a molecular state x , M independent b -step continuations give the Monte Carlo target

$$\hat{h}_M(x, z, b) = \frac{1}{M} \sum_{m=1}^M g_z(X_b^{(m)}), \quad X_b^{(m)} \sim R_\theta^b(\cdot | x). \quad (16)$$

When g_z is an indicator, this target estimates finite-budget reachability. For a general nonnegative desirability, it estimates expected future value. Training and validation are partitioned by molecular source so that states descended from the same source do not cross splits.

The exact backward values also satisfy the Bellman recursion in Equation (12), which provides an additional source of supervision when adjacent-budget trajectory pairs are available. We use this consistency relation as an auxiliary training signal for the reachability-valued controller, while controllers trained on soft expected-desirability targets are fit directly to their Monte Carlo future-value estimates. In both cases, h_ϕ is trained to approximate the backward quantity defining the exact

controller rather than an independent task-specific score. Architecture and training details for the respective future-value models are given in Appendices D.3 and D.4.

This distinction is important when terminal desirability is derived from a learned property model $\hat{F}_\psi$. The property model $\hat{F}_\psi(x)$ estimates properties of the current molecule, whereas $h_\phi(x, z, b)$ estimates the desirability reachable from that molecule after b further transitions of R_θ . Property learning therefore estimates where value may lie, while future-value learning propagates that value through the molecular process.

Replacing the exact backward value in Equation (13) yields the scalable controlled kernel

$$P_\phi(y \mid x, z, b) = \frac{R_\theta(y \mid x) h_\phi(y, z, b-1)}{\sum_{y' \in \mathcal{S}(x)} R_\theta(y' \mid x) h_\phi(y', z, b-1)}, \quad y \in \mathcal{S}(x), \quad (17)$$

whenever the denominator is positive. If $h_\phi = h$, Equation (17) recovers the exact finite-horizon transform of Theorem 3. With learned h_ϕ , approximation error can change how probability is allocated among executable successors, but cannot introduce a transition outside the support of R_θ . Exact terminal reweighting remains a guarantee of the exact h -transform and is not claimed for the learned approximation.

4.6 SAMPLING THE CONTROLLED MOLECULAR PROCESS

At inference, COMPOSE applies the controlled kernel in Equation (17) recursively over realized molecular states. Given the current molecule x , objective z , and remaining budget b , the executor enumerates the canonical successor set $\mathcal{S}(x)$. Then R_θ assigns goal-independent reference probability to these successors, and h_ϕ reweights them by their predicted remaining future value. After sampling and committing a successor, the resulting molecule becomes the next state and the budget is decremented. Recomputing the controlled law after every committed edit therefore yields a closed-loop molecular trajectory whose decisions depend on the state actually reached rather than on a trajectory fixed in advance.

When $\mathcal{S}(x)$ can be evaluated directly, we sample from the normalized kernel in Equation (17). The same row can alternatively be sampled by rejection whenever h_ϕ admits a finite envelope, since proposing from $R_\theta(\cdot \mid x)$ and accepting in proportion to h_ϕ recovers the controlled distribution conditional on acceptance. For objectives whose desirable futures carry little reference mass, direct sampling can allocate insufficient computation to useful trajectories. We instead use a Feynman–Kac sequential Monte Carlo construction. Particles propagate under the frozen reference process R_θ , and a transition $x \rightarrow y$ with b remaining steps receives incremental potential

$$G_b(x, y) = \frac{h_\phi(y, z, b-1)}{h_\phi(x, z, b)}. \quad (18)$$

Particles are resampled when effective sample size falls below $N/2$. When h_ϕ equals the exact backward value, these potentials recover the finite-horizon Doob-controlled path law of Section 4.4. With learned h_ϕ , the same value approximation acts as a twisting function for the reference particle system, and the resulting finite-particle procedure is an approximation whose target and boundary conditions are detailed in Appendix E.

For objectives defined by a target region $\mathcal{B}_z$, we stop prospectively at the first realized hit,

$$\tau = \min(\{t \leq K : X_t \in \mathcal{B}_z\} \cup \{K\}). \quad (19)$$

A reached molecule is returned when the target region is entered. If no hit occurs, the trajectory continues to its declared budget. Intermediate states are not retrospectively collected as additional samples, and one controlled trajectory produces one returned candidate. Objectives without a first-hit interpretation are sampled to the declared horizon.

4.7 PREFERENCE-CONDITIONED MULTI-OBJECTIVE CONTROL

The preceding construction applies to any nonnegative terminal desirability g_z . To control COMPOSE under competing molecular objectives, let $\hat{\mathbf{F}}_\psi(x) = (\hat{F}_{\psi,1}(x), \dots, \hat{F}_{\psi,m}(x)) \in [0, 1]^m$ denote

frozen property scores oriented so that larger values are preferred, and let $\lambda \in \Delta^{m-1}$ specify a preference over objectives. We define the weighted Chebyshev shortfall

$$L_\lambda(x) = \max_{i=1,\dots,m} \lambda_i (1 - \hat{F}_{\psi,i}(x)). \quad (20)$$

Unlike an additive scalarization, the maximum is determined by the most limiting preference-weighted objective, preventing improvement in one property from arbitrarily compensating for poor performance on another.

We convert this shortfall into the nonnegative terminal desirability required by the finite-horizon controller,

$$g_\lambda(x) = \exp\left(-\frac{L_\lambda(x)}{\tau_\lambda}\right), \quad \tau_\lambda = \text{Std}_{x \sim \mathcal{D}_{\text{train}}}[L_\lambda(x)] > 0. \quad (21)$$

The scale τ_λ is fixed from the property-supervision set before molecular optimization and is not fitted to generated candidates. Numerical centering used in the implementation leaves normalized controlled transitions unchanged and is detailed in Appendix D.4.

Setting $z = \lambda$ in Section 4.5 gives

$$h_\phi(x, \lambda, b) \approx \mathbb{E}_{R_\theta}[g_\lambda(X_b) \mid X_0 = x],$$

and Equation (17) then defines the corresponding preference-conditioned molecular process. Different values of λ therefore alter the desired tradeoff through the controller while leaving the learned molecular reference process R_θ fixed. Running the controller under multiple preferences yields preference-conditioned candidate sets, from which we report the nondominated subset. The scalarization itself carries no claim of Pareto optimality.

The same state-level formulation also permits objectives and admissibility requirements to change after generation has begun, as we describe next.

4.8 RETARGETING AND PATHWISE MOLECULAR CONSTRAINTS

Because control is conditioned on the current molecular state, COMPOSE can change either the design objective or the admissible transition set without retraining the molecular reference process. Suppose a trajectory controlled under objective z reaches the realized molecule x_τ with b transitions remaining, after which the objective changes to z' . Control resumes by evaluating the same kernel in Equation (17) from x_τ under the new goal context z' . The realized prefix $x_0 \rightarrow \dots \rightarrow x_\tau$ is retained, and no parameter of R_θ is updated.

Corollary 4 (Exact continuation after a goal change). *With exact backward values, applying Theorem 3 from (x_τ, b) with terminal desirability $g_{z'}$ realizes the $g_{z'}$ -tilted b -step terminal law of the reference process started at x_τ . This generally differs from specifying z' at the original source x_0 , because continuation preserves the molecular state actually reached, whereas restarting discards it. With learned h_ϕ , the same operation gives the corresponding approximate continuation.*

Hard statewise requirements can instead be enforced through executable support. Let $C : \mathcal{X} \rightarrow \{0, 1\}$ be a molecular predicate with $C(x_0) = 1$, and define the restricted edit set

$$\mathcal{A}_C(x) = \{a \in \mathcal{A}(x) : C(T(x, a)) = 1\}. \quad (22)$$

Applying the canonicalization and normalization of Section 4.3 to $\mathcal{A}_C(x)$ gives the constrained reference kernel R_θ^C . If $\mathcal{A}_C(x) = \emptyset$, the state is terminal under the constrained process. Otherwise every committed successor satisfies C by construction. Finite-horizon control then proceeds with R_θ^C in place of R_θ . Approximation error in h_ϕ may therefore misallocate probability among admissible successors, but cannot violate a constraint enforced through the support itself.

This construction applies to state predicates whose admissibility can be determined from the proposed molecular successor, and we refer to such constraints as *rule-closed*. Examples include protected scaffolds, forbidden substructures, and size or charge restrictions. Requirements that depend on accumulated history require an augmented state. For a sufficient statistic ξ_k , define

$$\tilde{X}_k = (X_k, \xi_k), \quad \xi_{k+1} = \Phi(\xi_k, X_{k+1}),$$

so that the augmented process remains Markov. Cumulative edit costs or other history-dependent requirements can then be represented explicitly rather than imposed through a non-Markov rejection rule.

Retargeting and pathwise restriction therefore modify different parts of the same frozen process. Retargeting changes the control context, whereas hard constraints change executable support. In both cases, the parameters of the learned molecular reference process remain fixed.

5 EXPERIMENTS

We evaluate COMPOSE from the learned molecular reference process through finite-horizon control, standard molecular editing, multi-objective design, and trajectory-level intervention.

5.1 COMPOSE LEARNS STATE-DEPENDENT PREFERENCES AMONG EXECUTABLE MOLECULAR EDITS

The executable rewrite system determines which molecular edits are possible, but not which should be preferred in a given molecular state. We therefore test whether the frozen reference law R_θ learns state-dependent transition structure beyond corpus-wide edit-family frequencies. On 3,545 held-out transitions with source molecules disjoint from reference-law training, we compare R_θ with an empirical-family baseline that uses training-set family frequencies while operating over the same canonical molecular successors. This holds executable support fixed and asks only whether learning improves how probability is distributed among the available next molecules.

R_θ reduces canonical-successor NLL from 5.37 to 3.90 nats relative to the empirical-family baseline (Figure 2(a)). A matched decomposition shows that 86.7% of this improvement is recovered by retaining empirical family frequencies and learning only which successor is preferred within the current state. Thus most of the learned signal is not which kinds of edits occur globally, but which executable molecular edit is appropriate for the molecule at hand. The rewrite system therefore defines what is possible, while R_θ supplies a learned, state-dependent preference over those possibilities.

This effect persists when evaluation is restricted to edit families with data-backed transition supervision, where R_θ retains a 1.06-nat advantage over empirical family frequencies (Appendix F.1). The stronger gains observed for synthetically supervised ring-edit families are therefore not required for the state-dependent preference claim. We use R_θ throughout the remaining experiments as a goal-independent reference law over fixed executable support, without interpreting its probabilities as physical kinetics or synthetic-route likelihood.

5.2 COMPOSE USES FUTURE REACHABILITY TO IMPROVE MULTI-STEP MOLECULAR EDITING

COMPOSE can evaluate a molecular edit by the futures it leaves reachable, rather than only by the molecule it produces immediately. Because control acts on canonical molecular successors and every committed successor is itself a complete molecule, each candidate edit defines a realized molecular intermediate from which the frozen reference process can be continued over the remaining budget. We isolate this capability through exact target recovery on 65 held-out source–target pairs with source molecules disjoint from development, each admitting a certified executable edit path within budget. Both policies operate over the same frozen R_θ , target, edit budget, and candidate successors. The local policy greedily selects the successor most similar to the target, whereas the future-aware rollout policy evaluates the continuations reachable over the remaining budget and overrides the local choice only when doing so yields a strict improvement. We independently verify the corresponding exact finite-horizon construction on an enumerable molecular state graph (Appendix F.2).

Accounting for future reachability changes which molecular edits succeed. Remaining-budget rollout recovers 40/65 targets, compared with 26/65 under local selection, rescuing 14 of the 39 local failures (+21.5 percentage points, [12.3, 33.5], Figure 2(b–c)). A deterministically selected rescue in Figure 2(b) illustrates the mechanism. The local policy chooses the higher-similarity successor, whereas a single future-aware override takes a lower-similarity edit that preserves a successful continuation and reaches the target within budget. Across the full panel, these rescues show that the value of an edit depends not only on the molecule it produces immediately, but on the molecular futures it leaves reachable.

At molecular scale, exhaustive evaluation of these continuations is expensive, so we use h_ϕ to prioritize which successors receive rollout evaluation. This retains the same aggregate recovery using only 34.6% of the continuation evaluations required by all-candidate lookahead (Figure 2(d)). We treat this result as a prioritization diagnostic rather than a standalone evaluation of h_ϕ , because the two procedures do not recover exactly the same targets, and exact-target similarity uses information unavailable in ordinary property optimization. Section 5.4 removes this privileged target information and tests whether the same future-reachability principle improves target-free multi-objective molecular design.

5.3 COMPOSE PERFORMS STRONGLY ON STANDARD SOURCE-CONDITIONED MOLECULAR EDITING

We next ask whether the complete COMPOSE system is competitive on a standard molecular-editing task. We use the ZINC-250k similarity-constrained QED benchmark reported by GrIDDD (Ninniri et al., 2025). Source molecules begin with QED in $[0.7, 0.8]$, and a source is solved if at least one returned molecule reaches $\text{QED} \geq 0.9$ while retaining Morgan-fingerprint Tanimoto similarity of at least 0.4. The reference law R_θ is unchanged from Sections 5.1 and 5.2, and only the QED future-value controller is trained for this task, using sources disjoint from evaluation. Each candidate slot corresponds to one controlled trajectory. Successful trajectories terminate prospectively upon first entering the target region, while trajectories that never reach the region remain failures rather than being resampled.

Method	Sources	K	QED success ($\uparrow$)	Diversity ($\uparrow$)
JT-VAE (Jin et al., 2018)	800	20	8.8%	–
CG-VAE (Liu et al., 2018)	800	20	4.8%	–
GCPN (You et al., 2018)	800	20	9.4%	0.216
GrIDDD (Ninniri et al., 2025)	800	20	45.1%	0.283 [†]
COMPOSE	128*	12*	54.7%*	0.393*

Table 1: **Source-conditioned editing on the Jin/ZINC-250k similarity-constrained QED benchmark.** Published baseline values are reproduced from the comparator set reported by GrIDDD. K denotes candidates returned per source, and success requires $\text{QED} \geq 0.9$ and Tanimoto similarity ≥ 0.4 . *COMPOSE is currently evaluated on a separate 128-source held-out panel with $K = 12$, whereas published rows use the official 800 sources with $K = 20$, so starred values are shown as benchmark context rather than a direct numerical ranking. [†]GrIDDD reports diversity without specifying its evaluator implementation, whereas COMPOSE uses the released Jin evaluator. Diversity details and the broader historical comparison are reported in Appendix G.

COMPOSE achieves strong source-conditioned editing performance under the standard similarity-constrained QED criterion. On the held-out evaluation, COMPOSE solves 70/128 sources (54.7%) within 12 returned candidates and maintains substantial within-source molecular diversity (0.393 mean pairwise Tanimoto distance, Figure 3a), indicating that benchmark success is not driven by collapse to a narrow set of edits. Success increases steadily with candidate allowance rather than depending on a single operating point. Table 1 places this result alongside published methods evaluated under the same benchmark criterion, including GrIDDD at 45.1% success. Because the current COMPOSE evaluation uses a different source cohort and candidate allowance, these values are reported as benchmark context rather than a direct numerical ranking. Internal inference costs are not matched across methods, since the benchmark constrains the number of returned molecules rather than the successor evaluations performed to produce them.

5.4 FUTURE-AWARE CONTROL IMPROVES PARETO EXPLORATION UNDER COMPETING OBJECTIVES

Section 5.2 showed that remaining-budget information can improve molecular decisions when the desired endpoint is known. We now ask whether the same principle improves multi-objective design when no target molecule is specified. We optimize JNK3 and GSK3 β across five preference directions following the InversionGNN setting (Niu et al., 2025). The immediate and future-aware COMPOSE controllers share the same frozen R_θ , objective-blind initialization bank, property model, executable edit sets, preferences, and evaluation budget. They differ only in whether a candidate is valued by

its properties at the next molecule, $g_\lambda(y)$, or by the preference-weighted futures reachable from it, $h_\phi(y, \lambda, b - 1)$. Average Pareto score (APS) is the primary endpoint, while hypervolume, novelty, and diversity are reported in Table 2 as complementary measures of front quality.

Method	APS ($\uparrow$)	HV ($\uparrow$)	Novelty ($\uparrow$)	Top- K diversity ($\uparrow$)
InversionGNN, matched rerun	XXX	XXX	XXX	XXX
COMPOSE, immediate preference	XXX	XXX	XXX	XXX
COMPOSE, future-aware	XXX	XXX	XXX	XXX

Table 2: **Two-objective JNK3/GSK3 β optimization from a common frozen initialization bank.** All rows use the same evaluation budget and preference set. Published InversionGNN results obtained under a different initialization protocol are reported as context in Appendix G.3 and are not pooled with the matched rerun.

Future-aware control achieves an APS of **XXX**, compared with **XXX** under immediate control, an improvement of **XXX** (Table 2, Figure 3b). Hypervolume changes from **XXX** to **XXX**, while novelty and diversity remain **XXX**, indicating [if supported: that the improvement is not obtained by collapsing onto a narrow region of molecular or objective space]. Because both COMPOSE arms see the same candidate molecules and identical property predictions, this comparison isolates the effect of propagating preference through the molecular futures induced by R_θ . The gain therefore extends the mechanism of Section 5.2 from known-target recovery to target-free multi-objective design.

Under the same initialization, preference set, and evaluation budget, the matched InversionGNN rerun obtains an APS of **XXX** (Table 2), providing an external measure of multi-objective task competence. This cross-method comparison is not used to attribute the difference to future-aware control, since the methods do not share a learned property predictor or internal search procedure. These results show that the same frozen molecular process can be redirected toward distinct property trade-offs. We next ask whether this flexibility persists after molecular optimization is already underway and the objective itself changes.

5.5 COMPOSE’S REALIZED MOLECULAR STATES PRESERVE PROGRESS WHEN OBJECTIVES CHANGE

Section 5.4 showed that the same frozen molecular process can be steered toward different property preferences when those preferences are specified in advance. Design priorities, however, can change after optimization is already underway, for example as new measurements introduce liabilities or additional requirements. COMPOSE is naturally suited to this setting because its reference process R_θ is goal-independent and every realized state is itself a complete molecule. A newly specified objective can therefore be applied directly to the molecule already reached. We ask whether this capability is practically useful. Does molecular progress accumulated before an objective change provide a better starting point than beginning again from the original molecule?

We test this on 40 source-disjoint held-out molecules under potency-first and developability-first histories. Each source is edited for three steps before the eventual joint objective B is specified, so the committed prefix cannot anticipate the new goal. After the switch, continuing from the realized state x_3 and restarting from the original molecule x_0 receive the same B , the same three-edit remaining budget, and the same frozen molecular process. This matched comparison isolates whether the molecular state reached before the switch retains useful progress for the newly specified objective.

Comparison	Potency-first	Developability-first
State reuse (continue x_3 vs. restart x_0)	+0.302 [0.186, 0.413]	+0.176 [0.069, 0.283]
Retargeting (new vs. old objective)	+0.748 [0.608, 0.891]	+1.462 [1.240, 1.691]

Table 3: **Reusing realized molecular states after an objective switch.** Entries are paired changes in the post-switch objective, and positive values favor the first strategy. State reuse is the primary comparison, and intervals are source-level bootstrap intervals. The complete record is given in Appendix F.5.

Continuing from x_3 improves the post-switch objective over restarting from x_0 by 0.302 [0.186, 0.413] after potency-first optimization and 0.176 [0.069, 0.283] after developability-first optimization (Table 3, Figure 3c). The benefit is not universal across sources, indicating that state

reuse is an empirical advantage rather than a consequence of the controller construction. Retargeting also substantially outperforms continuing under the previous objective, confirming that the subsequent trajectory responds to the newly specified goal. Thus, when design priorities change, COMPOSE can redirect molecular progress already realized under an earlier objective rather than discarding that progress and beginning again from the original molecule.

5.6 COMPOSE PREVENTS CONSTRAINT VIOLATIONS HIDDEN BY ENDPOINT-ONLY MOLECULAR OPTIMIZATION

Section 5.5 showed that realized molecular states can remain useful when design objectives change. Their explicit representation provides a second advantage, in that constraints can govern which molecular states the optimization process is allowed to visit, rather than only which molecule it ultimately returns. This distinction matters whenever intermediate designs must remain within a predictor’s applicability domain, preserve a required molecular feature, or satisfy a prescribed physicochemical range. Endpoint-only optimization cannot certify such a requirement from the final molecule alone, whereas COMPOSE can impose it directly by restricting the executable successor support at every realized state.

We test whether this distinction is consequential using a fixed cLogP corridor C while optimizing an independent DRD2 objective over six edits. Endpoint-only control requires only the returned molecule to lie within C , whereas COMPOSE’s pathwise controller removes successors outside C before a transition is sampled. The two arms otherwise share the same R_θ , sources, objective, horizon, and evaluation budget. Because zero pathwise violations follow from support restriction by construction, the empirical question is not whether the mask works, but whether endpoint-only optimization actually hides meaningful violations that make the stronger guarantee worth having.

Endpoint-only feasibility conceals substantial pathwise violations. On the current evaluation, 14/24 (58.3%) of endpoint-feasible trajectories leave C at least once before returning. These excursions extend a median 0.69 cLogP units beyond the admissible region and persist for four of six committed edits (Figure 3d). Endpoint evaluation can therefore certify a final molecule while overlooking violations across much of the optimization trajectory.

COMPOSE prevents these hidden violations by restricting the molecular states reachable during optimization. Under future-aware pathwise control, normalized terminal DRD2 utility changes by only -0.023 $[-0.182, 0.134]$ relative to endpoint-only control, remaining within the preregistered tolerance. Thus the pathwise guarantee is not merely formal, since COMPOSE can enforce a requirement throughout molecular optimization while retaining comparable endpoint utility. This capability follows from controlling an explicit process over complete molecular states, in which constraints can act on the trajectory itself rather than being checked only after generation is complete. The complete developmental record, the support-viability audit and the confirmation preregistration are given in Appendix F.6.

6 DISCUSSION AND LIMITATIONS

Taken together, our results support a view of molecular design as control of a learned executable process rather than endpoint generation followed by scoring or repair. COMPOSE separates which molecular transitions are *possible*, which legal transitions are *plausible*, and which plausible futures serve the current *purpose*. The learned reference law captures state-dependent transition preferences beyond executable support alone, while finite-horizon control can redirect this frozen law according to future reachability. Because every committed state is itself a molecule, control can also be reapplied to states already reached and constraints can act on the trajectory rather than only its endpoint. Variable-size editing, stochastic molecular processes, and h -transforms are not individually new. The distinction is that COMPOSE brings them together in a common molecular transition process whose executable support and learned plausibility remain separate from task-specific purpose.

This decomposition also makes the limitations explicit. Control cannot create a useful continuation that is absent from the executable support or assigned negligible probability by the reference process. Exact finite-horizon guarantees apply under exact backward values, whereas molecular-scale control uses an approximate learned h_ϕ . Improved task performance therefore demonstrates useful future-aware steering, not exact conditioning at scale. Property-directed control additionally inherits errors

702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755

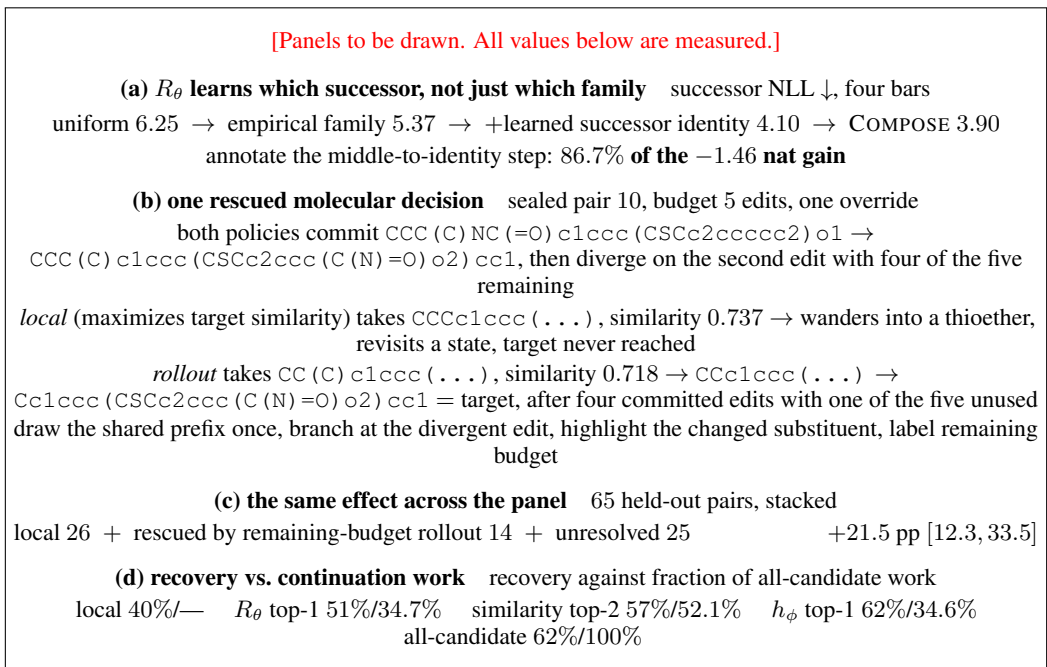


Figure 2: **Learning and controlling an executable molecular process.** (a) On the same canonical successor support, the learned reference law assigns higher probability to the realized successor than either unlearned law, and most of that gain is recovered by learning successor identity while retaining empirical family frequencies. (b) A single rescued decision, selected deterministically as the lowest-index rescue at the median certified path length. The local policy here selects the successor most similar to the target, and the two policies agree until four of the five edits remain. The rollout policy then declines that locally preferred successor in favour of a less similar one whose continuation reaches the target with an edit to spare, while the local branch does not recover it. One override changes the endpoint. (c) Across the panel, remaining-budget evaluation rescues 14 of the 39 targets local selection misses. Because the local action is retained by construction, regressions are excluded and the rescue count is the informative quantity. (d) Goal-aware prioritization concentrates continuation work. h_ϕ top-1 matches all-candidate recovery at about a third of the continuations. The recovered sets differ, and exact-target similarity has privileged information unavailable in ordinary property optimization.

and extrapolation from the property models defining desirability. Finally, the declared state space remains a 2D molecular representation, where connectedness and valence admissibility do not imply stereochemical correctness, conformational plausibility, synthesizability, stability, or biological activity. These properties require richer representations or independent downstream evaluation.

The natural next step is therefore to improve the *molecular process itself*, not only the controller placed on top of it. Richer executable state spaces and reference models could expand which chemically meaningful futures are reachable, while new measurements could update which of those futures are preferred without redefining the underlying transition process. More broadly, COMPOSE suggests that when a domain admits meaningful executable transformations between valid states, those transformations can form the native event language of a learned stochastic process, with generation and control acting on the same evolving object.

REFERENCES

- Anonymous. pCoMole: Pareto-constrained molecule editing with discrete flows. OpenReview, 2026. OpenReview submission <https://openreview.net/forum?id=LpttbUObQL>.
- Andrew Campbell, Jason Yim, Regina Barzilay, Tom Rainforth, and Tommi Jaakkola. Generative flows on discrete state-spaces: Enabling multimodal flows with applications to protein co-design. In *International Conference on Machine Learning (ICML)*, 2024.
- Tong Chen, Yinuo Zhang, and Pranam Chatterjee. AReURDi: Annealed rectified updates for refining discrete flows with multi-objective guidance. *arXiv preprint arXiv:2510.00352*, 2025.

756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809

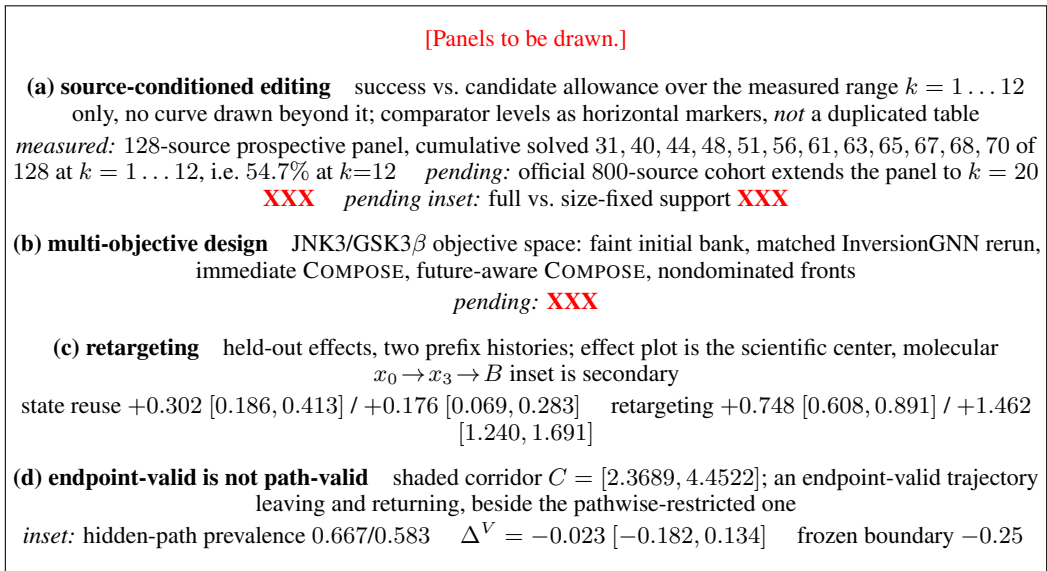


Figure 3: **Molecular design with a frozen reference process.** One reference law across all four panels. (a) Standard similarity-constrained editing as candidate allowance grows. (b) Preference-conditioned multi-objective design in the objective space itself rather than as a scalar. (c) Continuing from the realized molecule is better than restarting from the source once the objective changes. (d) Endpoint filtering accepts trajectories that left the admissible corridor, whereas support restriction makes those states unreachable by construction, which is definitional and not an empirical superiority statistic. Panel (d) values are developmental pending the frozen 48-source confirmation.

Tianfan Fu, Cao Xiao, Xinhao Li, Lucas M. Glass, and Jimeng Sun. MIMOSA: Multi-constraint molecule sampling for molecule optimization. In *AAAI Conference on Artificial Intelligence*, 2021.

Shrey Goel and Pranam Chatterjee. Minimal-action discrete Schrödinger bridge matching for peptide sequence design. *arXiv preprint arXiv:2601.22408*, 2026.

Rafael Gómez-Bombarelli, Jennifer N. Wei, David Duvenaud, José Miguel Hernández-Lobato, Benjamín Sánchez-Lengeling, Dennis Sheberla, Jorge Aguilera-Iparraguirre, Timothy D. Hirzel, Ryan P. Adams, and Alán Aspuru-Guzik. Automatic chemical design using a data-driven continuous representation of molecules. *ACS Central Science*, 4(2):268–276, 2018.

Marton Havasi, Brian Karrer, Itai Gat, and Ricky T. Q. Chen. Edit flows: Flow matching with edit operations. *arXiv preprint arXiv:2506.09018*, 2025.

Peter Holderrieth, Marton Havasi, Jason Yim, Neta Shaul, Itai Gat, Tommi Jaakkola, Brian Karrer, Ricky T. Q. Chen, and Yaron Lipman. Generator matching: Generative modeling with arbitrary markov processes. In *International Conference on Learning Representations*, 2025.

Cristian Jensen, Luca Schaufelberger, Riccardo De Santi, Kjell Jorner, and Andreas Krause. Value matching: Scalable and gradient-free reward-guided flow adaptation. In *International Conference on Learning Representations (ICLR)*, 2026. OpenReview <https://openreview.net/forum?id=7iXt44Actj>.

Jan H. Jensen. A graph-based genetic algorithm and generative model/monte carlo tree search for the exploration of chemical space. *Chemical Science*, 10(12):3567–3572, 2019.

Wengong Jin, Regina Barzilay, and Tommi Jaakkola. Junction tree variational autoencoder for molecular graph generation. In *International Conference on Machine Learning (ICML)*, 2018.

Wengong Jin, Kevin Yang, Regina Barzilay, and Tommi Jaakkola. Learning multimodal graph-to-graph translation for molecular optimization. In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1812.01070.

- Wengong Jin, Regina Barzilay, and Tommi Jaakkola. Hierarchical generation of molecular graphs using structural motifs. In *International Conference on Machine Learning (ICML)*, volume 119 of *Proceedings of Machine Learning Research*, pp. 4839–4848, 2020.
- Seul Lee, Karsten Kreis, Srimukh Prasad Veccham, Meng Liu, Danny Reidenbach, Yuxing Peng, Saeed Gopal Paliwal, Weili Nie, and Arash Vahdat. GenMol: A drug discovery generalist with discrete diffusion. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 33205–33226, 2025. URL <https://proceedings.mlr.press/v267/lee25o.html>.
- Qi Liu, Miltiadis Allamanis, Marc Brockschmidt, and Alexander L. Gaunt. Constrained graph variational autoencoders for molecule design. In *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 7806–7815, 2018.
- Matteo Ninniri, Marco Podda, and Davide Bacciu. Graph diffusion that can insert and delete. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2506.15725.
- Yifan Niu, Ziqi Gao, Tingyang Xu, Yang Liu, Yatao Bian, Yu Rong, Junzhou Huang, and Jia Li. InversionGNN: A dual path network for multi-property molecular optimization. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2503.01488.
- Sophia Tang, Yinuo Zhang, and Pranam Chatterjee. PepTune: De novo generation of therapeutic peptides with multi-objective-guided discrete diffusion. In *International Conference on Machine Learning (ICML)*, 2025. arXiv:2412.17780.
- Clément Vignac, Igor Krawczuk, Antoine Siraudin, Bohan Wang, Volkan Cevher, and Pascal Frossard. Digress: Discrete denoising diffusion for graph generation. In *International Conference on Learning Representations*, 2023.
- Yutong Xie, Chence Shi, Hao Zhou, Yuwei Yang, Weinan Zhang, Yong Yu, and Lei Li. MARS: Markov molecular sampling for multi-objective drug discovery. In *International Conference on Learning Representations (ICLR)*, 2021.
- Seokwon Yoon, Youngbin Choi, Seunghyuk Cho, Seungbeom Lee, MoonJeong Park, and Dongwoo Kim. Routing by reaching: Composition of pre-trained GFlowNets for multi-objective generation. In *International Conference on Machine Learning (ICML)*, 2026. arXiv:2602.21565.
- Jiaxuan You, Bowen Liu, Rex Ying, Vijay Pande, and Jure Leskovec. Graph convolutional policy network for goal-directed molecular graph generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 6410–6421, 2018. arXiv:1806.02473.
- Zhenpeng Zhou, Steven Kearnes, Li Li, Richard N. Zare, and Patrick Riley. Optimization of molecules via deep reinforcement learning. *Scientific Reports*, 9(1):10752, 2019.

OVERVIEW OF APPENDIX

This appendix provides the formal, implementation, and experimental details supporting the main text. Appendix A specifies the executable molecular state space, rewrite semantics, and canonical molecular-successor representation. Appendix B gives the complete theoretical results underlying support preservation and finite-horizon control. Appendices C to E describe transition supervision, model training, and controlled sampling. Appendix F gives the experimental details and extended results for each experiment in the order it appears in Section 5, Appendix G documents alignment with external benchmark protocols, and Appendix H records the software environment, randomization scheme, and compute accounting.

A MOLECULAR STATE SPACE AND EXECUTABLE EDITS

The support-level claims of COMPOSE are defined with respect to an explicit molecular state space rather than post-hoc sanitization. This section specifies that state space and the rewrite contracts that induce its transition graph. The *semantic state* is the active molecular graph on which connectivity,

valence, ring perception, and canonical identity are evaluated, and we distinguish it from any padded tensor representation used by the implementation. We then define the current operator family, its exact legality checks, and the pushforward from executable rewrite marks to canonical molecular successors. These definitions determine what transitions are possible before either R_θ or a task-specific controller assigns probability to them.

A.1 MOLECULAR STATE SPACE

States carry at most **XXX** heavy atoms over **XXX** neutral (element, valence) classes drawn from a declared organic element set, a vocabulary admitting **XXX** of the benchmark lead molecules. Formal charge is an input but not an output dimension, so charge is carried rather than designed. Stereochemistry, isotopes and radicals are not represented.

Cardinality is a property of the active molecular graph rather than of a fixed tensor allocation. Writing $\mathcal{X}_n$ for the states with n heavy atoms, the state space is the disjoint union of the strata, and a birth or death moves a trajectory between strata by creating or destroying a typed graph entity that participates in valence, connectivity, and ring perception. This is not the activation or deactivation of a padded coordinate, and the distinction is what makes the process trans-dimensional in the sense used throughout the paper.

A.2 REWRITE OPERATORS

The main text depends only on the legal edit set (5) and never on which families populate it. This section records the families used, so that the edit set is reproducible. We use paper-facing operator names for executable rewrite semantics and model-family names when reporting learned-family statistics, and Table 4 gives the fixed mapping between them.

`cycle_close` inserts one bond between two nonbonded atoms and, because every non-null state is connected, raises graph cycle rank by one, while `cycle_open` deletes one non-bridge cycle edge and lowers it by one. At the untyped graph level every connected graph is a spanning tree plus its non-tree edges, so inserting those edges one at a time realizes the abstract cyclic topology while preserving connectivity. This is a statement about graphs. It does not prove that the molecular grammar admits a valence-valid path to every chemically labelled ring system, nor that such a path fits a finite edit budget. Both are measured questions and neither is assumed anywhere in the main text.

A.3 CANONICAL MOLECULAR SUCCESSORS

Distinct executable marks can produce the same molecule, so the law the controller acts on is the pushforward of the mark law through the executor, aggregated at canonical molecular identity as in (9)–(11). Aggregation is what makes every downstream quantity independent of how many ways the implementation can name a molecule, and Appendix B.2 establishes the precise invariance this buys and its exact scope.

B THEORETICAL RESULTS

Results are ordered as a reader needs them, with state-space closure first, then the canonical-successor quotient and its invariance, then the finite-horizon control results that depend on both. Each statement is repeated here with its assumptions so that the appendix stands alone.

B.1 EXECUTABLE-SUPPORT CLOSURE

We restate each result with its assumptions before proving it, so that this appendix can be read independently of the main text.

Proposition 5 (Closure of executable support, restatement of Proposition 1). *Membership in $\mathcal{X}$ requires exactly four conditions. Node and edge attributes lie in the declared vocabularies, every atom satisfies its declared valence constraints, the graph is connected, and its heavy-atom count lies in $\{1, \dots, n_{\max}\}$. Suppose each admitted edit keeps every created or modified attribute within the*

Table 4: The rewrite families, by action on the strata. Each family is a typed graph rewrite with structural and chemical application conditions. The executor follows a propose–validate–commit discipline and exposes no transition whose successor fails a guard. Names here are the paper-facing operator names. The implementation additionally carries model-family identifiers, which appear in per-family results, where `cycle_close` is recorded as `cycle_insert`, `cycle_open` as `cycle_attach`, and `ring_aromaticity_restate` as `ring_system_restate`. The remaining five names coincide in both registers. Per-family results in Appendix F.1 are reported under the model-family identifiers.

Family	Cardinality	Structural / chemical condition
<code>atom_insert</code>	birth $\mathcal{X}_n \rightarrow \mathcal{X}_{n+1}$	empty capacity; a null source or exactly one occupied anchor; allowed atom state; for a bonded birth, allowed bond order and feasible endpoint valence
<code>atom_delete</code>	death $\mathcal{X}_n \rightarrow \mathcal{X}_{n-1}$	removable atom with connected remainder; neighbour hydrogen compensation; successor sanitizes
<code>atom_restate</code>	same	occupied site; joint element, valence-class and implicit-hydrogen state allowed; successor differs from the source
<code>bond_reorder</code>	same	existing edge; joint bond-order and hydrogen update feasible
<code>bond_reroute</code>	same (reroute)	the cut edge is a bridge; the relocated fragment is a pendant subtree; the new endpoint lies outside it; cut and attach commit in one transaction so no disconnected state is ever exposed
<code>cycle_close</code>	same (cyclize)	absent edge between two occupied atoms with valence headroom; admitted insertion raises graph cycle rank by one
<code>cycle_open</code>	same (decyclize)	existing edge that is not a bridge; endpoint hydrogen compensation; successor sanitizes
<code>ring_aromaticity_restate</code>	same	a matched cyclic system; a coordinated set of bond-order changes validated by the executor as one transaction
whole-ring growth, whole-ring deletion	multi-atom macros	disabled ; a macro must earn its place by a measured path-efficiency gain, never by assumption

declared vocabularies, preserves the declared valence constraints at every atom it modifies, leaves untouched atoms unchanged, respects the heavy-atom bound $n_{\max}$, and is committed only when its successor is connected. Then $\mathsf{T}(x, a) \in \mathcal{X}$ for every $x \in \mathcal{X}$ and $a \in \mathcal{A}(x)$, and consequently every state reached by any stochastic process supported on $\{\mathcal{A}(x)\}_{x \in \mathcal{X}}$ remains in the declared molecular state space.

Proof. Induct on committed events. The initial state lies in $\mathcal{X}$. If $x_n \in \mathcal{X}$, the process has support only on $a \in \mathcal{A}(x_n)$, and by (5) every such mark executes to a state that passed the executor’s guards. Those guards are exactly the four membership conditions above, checked before the edit is committed, so the successor lies in $\mathcal{X}$. The same induction applies verbatim to the controlled process of (17), because the controlled off-diagonal mass is a pointwise product of a nonnegative factor with a base probability and therefore vanishes wherever the base probability vanishes, and no controller creates support. $\square$

B.2 CANONICAL SUCCESSOR REPRESENTATION

Definition 6 (Successor-preserving, rate-preserving re-encoding). Two mark systems on the same state space are so related if for every x there is a surjection φ_x from the marks of one onto the marks of the other with matching executed successors, so that marks may be split or merged only *within* a successor fiber and never across fibers, and if the aggregate rate of every successor fiber agrees.

Proposition 7 (Encoding invariance, and its exact scope). *The following hold under Definition 6 and a common state-level productive-commit predicate. (i) the state-level generators coincide, hence so do the total hazards, the molecular laws, the reference edit law R_θ of (11) wherever defined, and every time marginal. (ii) for any controller that is fiber-constant, meaning a function of (x, y) rather than of the mark, the controlled kernels coincide. (iii) hard state predicates and the transform of Theorem 3 are fiber-constant by construction, so the values h_b and the controlled kernels are identical under any such re-encoding.*

Proof. (i) The fibers correspond under φ_x and partition the applicable set, so fiber sums agree successor by successor. Summing over successors gives equality of total hazards without further hypothesis, which answers the natural worry that a refinement, which renormalizes the mark law over a larger set, could perturb the hazard. It cannot, because the hazard is the sum of the aggregates. Equality of generators plus uniqueness of the finite-state forward solution gives equality of all marginals, and dividing the aggregate by the hazard gives equality of R_θ on every row with positive mass. (ii) A fiber-constant controller is a function $\Psi(x, y)$, so it multiplies two equal aggregates and yields equal controlled rates, and the diagonal is determined by the row. (iii) h_b is built by backward recursion from kernels that are invariant by (i), so h_b is invariant by induction, and the ratio $h_{b-1}(y)/h_b(x)$ is fiber-constant. $\square$

Remark 8 (Scope of encoding invariance). Proposition 7 applies to successor-preserving refinements and to controllers defined on molecular successors. It does not imply invariance of arbitrary mark-level control.

Merging marks *across* successor fibers is excluded by Definition 6 and is ill-posed rather than merely lossy, since it would stop the executor being a function of the merged mark. And mark-level controllers are not invariant in general. If a successor is reachable by four symmetric aliases of rate $1/4$ under one encoding and by a single mark of rate 1 under another, a mark-level power tilt $\Psi \propto q(a)^{\beta-1}$ assigns it controlled mass $4 \cdot (1/4)^\beta$ versus 1, which differ for every $\beta \neq 1$. Top- k and nucleus truncation over marks, and per-family multipliers, fail for the same reason. A family multiplier is invariant exactly when it is constant across every alias of every successor fiber it touches. Every controller in this paper is written over molecules. The proposition concerns the pushed-forward process, not the training representation. Mark-level supervision is generally not invariant to within-fiber refinement, which is why teacher supervision aggregates the successor fiber.

B.3 RELATION TO GENERATOR MATCHING

Section 4.2 trains R_θ by the productive-successor likelihood (8). This section records the exact relationship between that objective and generator matching (Holderrieth et al., 2025), and the two respects in which they differ.

Write the productive rate at a state as $Q^+(y | x, t) = \Lambda(x, t) P(y | x, t)$, separating a total productive exit rate from a normalized law over canonical molecular successors. For the Bregman divergence generated by $\varphi(u) = u \log u - u$, a direct computation gives, for each (x, t) ,

$$D_\varphi(Q^{+\star}, Q_\theta^+) = \underbrace{\Lambda^\star \text{KL}(P^\star \| R_\theta)}_{\text{conditional-successor block}} + \underbrace{\left(\Lambda^\star \log \frac{\Lambda^\star}{\Lambda_\theta} - \Lambda^\star + \Lambda_\theta \right)}_{\text{productive-hazard block}}, \quad (23)$$

where the second term is the same scalar divergence applied to the exit rates. The two blocks are variationally separable, where the first depends on θ only through R_θ and the second only through Λ_θ . Minimizing over the successor law therefore leaves the hazard block untouched, and conversely.

COMPOSE optimizes an empirical version of the conditional-successor block, and differs from the generator-matching population objective in exactly two respects. First, the hazard block is never optimized. The compiled supervision carries no rate magnitudes, since every stored transition is registered with unit teacher rate, which enters training only as a nonterminality indicator, so no target for Λ_θ exists in the corpus. Under that unit-rate convention $\Lambda^\star$ is constant and the weight it contributes to the first term is absorbed into a positive constant independent of θ . The reported results never read an exit-rate magnitude, and fixed-budget editing consumes only the embedded successor kernel, on which a constant rescaling of Q^+ has no effect.

Second, the expectation is taken under a different measure. Generator matching integrates the per-state divergence against the state–time marginal of the conditional-process mixture. COMPOSE instead draws training rows under a declared editing measure that is weighted across operator families by design, so that a family’s exposure reflects the reference law the model is meant to estimate rather than the incidental composition of the compiled corpus, and some strata are assigned zero draw. No importance correction is applied, and on zero-draw strata none could be. Consequently (23) holds pointwise in (x, t) , and (8) is the conditional-successor block of that decomposition, but the training functional is not the generator-matching population objective. We therefore describe the relationship as a decomposition that identifies which factor R_θ estimates, not as an objective identity.

B.4 FINITE-HORIZON CONTROL

Theorem 9 (Finite-horizon molecular control, restatement of Theorem 3). *Let R_θ define the reference path measure over K committed transitions from an initial molecule x_0 , with trajectories terminating before K assigned zero continuation mass, let $g_z \geq 0$ be a finite terminal desirability, and let h_b be given by the backward recursion (12). Wherever those backward values are positive, the controlled kernels $R_\theta^{z,b}$ of (13) are normalized, introduce no transitions outside the support of R_θ , and after K controlled edits realize the terminal law (14).*

Proof. (i) $\sum_y R_{\theta,b}(x, y) h_{b-1}(y) = h_b(x)$ by (12), so the ratios in (13) sum to one. (ii) Multiplication by $h_{b-1}(y)$ cannot make a zero base probability positive, and the supports are equal at a row exactly when no base-supported successor has zero value. (iii) For a path $x_0, \dots, x_K$ of positive base probability, the controlled path probability is the base path probability times $\prod_k h_{K-k-1}(x_{k+1})/h_{K-k}(x_k)$, and the interior factors telescope, leaving $h_0(x_K)/h_K(x_0) = g_z(x_K)/h_K(x_0)$. Every path to a fixed endpoint is therefore reweighted by the same factor, and summing over paths gives (14). The normalizer is the full-budget value at the start. Finally, for the continuation and horizon-specificity claims stated after Theorem 3, apply the same telescoping to the chain started at x with horizon b' and desirability $g_{z'}$. The marginal at $j < K$ is an h_{K-j} -tilt rather than a g_z -tilt, which is why early stopping does not inherit (14) and why continuation is not the same law as using $g_{z'}$ from the first edit. $\square$

B.5 RETARGETING AND PATHWISE SUPPORT

Retargeting is Theorem 3 applied from a realized state. Instantiating the theorem at (x_τ, b) with desirability $g_{z'}$ gives exactly the $g_{z'}$ -tilt of the reference law started at x_τ with budget b , which is in general not the law obtained by specifying z' at the source lead, and the transform applies because x_τ is itself a molecule by Proposition 1, so $g_{z'}$ is decidable on it.

A rule-closed predicate C restricts the legal edit set to $\mathcal{A}_C(x)$ as in (22). Because the restricted edit set is again a set of executable marks into $\mathcal{X}$, the closure argument above applies unchanged to the restricted process, Theorem 3 holds on it, and C holds at every committed molecule by induction rather than by filtering at the end. Cumulative requirements are not predicates on the current molecule, and augmenting the state with a running statistic $\xi_{k+1} = \Phi(\xi_k, X_{k+1})$ restores the Markov property on the augmented chain, and the same results then apply to $\tilde{X}_k = (X_k, \xi_k)$.

C DATA AND TRANSITION SUPERVISION

A reader should be able to answer, from this section alone, what evidence taught R_θ to prefer one legal transition over another. That question has two parts, namely which molecules the transitions were drawn from and how each transition was constructed. The second part is not uniform across the edit space, and it bounds how the strongest per-family results in Appendix F.1 may be read.

C.1 MOLECULAR DATA AND PREPROCESSING

The editing corpus was derived from a fixed 500,000-molecule GuacaMol source pool by compiling executable local transitions, real-endpoint analogue relations, and reversible synthetic walks. COMPOSE was not trained as an unconditional distribution model over that pool. The learned object is $R_\theta(y | x)$, a transition law over source–successor edits, not a density $p_\theta(x)$ over molecules.

The source pool is identified by the recorded dataset namespace and its SHA-256, and independently corroborated by the molecular statistics of the frozen training sources. Measured on 4,000 randomly sampled training sources, the element distribution is exactly the declared organic vocabulary, with carbon at 0.748, nitrogen and oxygen at 0.105 each, sulfur at 0.015, fluorine at 0.012, chlorine at 0.009, bromine at 0.003, iodine and phosphorus at 0.002, and boron at 0.001. The median heavy-atom count is 26, median molecular weight 371.6 and ninetieth percentile 499.3, median ring count 3, median aromatic-ring count 2, and only 1.6% acyclic. Because the training receipts do not bind the original raw asset end to end, we treat this provenance link as strongly supported rather than cryptographically complete. The frozen compiled corpus itself is exactly identified by its dataset, library, packed-store and reserve digests. The benchmark sources of Appendix G.1 are drawn from a different pool.

C.2 TRANSITION CONSTRUCTION

Transitions enter the corpus through several lanes, which are near family-pure with lane-homogeneous chunks. The frozen structural gate records 1,803,032 training transitions passing with zero violations. A metadata scan of every training chunk attributes `cycle_insert` (122,183 transitions across 102 chunks), `cycle_attach` (84,940 across 102) and `ring_system_restate` (16,380 across 29) entirely to the reversible synthetic-walk lane, with zero transitions in any of the four data-backed lanes. In the pinned distribution-matched training subset that lane contributes 23,100 of 145,756 transitions, or 15.8%, against 84.2% data-backed. By contrast `bond_reroute` and `atom_restate` each appear in two lanes and therefore do have a real-chemistry source. The analogue lane draws on a frozen molecular-matched-pair pool of 344,153 records bound by a recorded contract hash, whose own upstream molecular source is not fully documented.

The consequence is a scope limit rather than a defect, and it is applied throughout. Evidence from the ring and cycle families supports the claim that the process recovers legal ring edits under synthetic perturbation, and does not support a claim of data-backed ring editing or scaffold hopping. Lifting that scope requires a data-backed ring-transformation lane, which the current corpus does not contain.

C.3 SUPERVISION PROVENANCE AND DATA SPLITS

Every result in Section 5 is bound to artifacts frozen before it. The reference edit law was never retrained for any experiment in this paper, and the controller head evaluated on the prospective panel was verified byte-identical by hash before and after an unrelated training session. The three source panels (development, prospective validation, and the official cohort) were verified pairwise disjoint, with the official cohort and the development panel excluded from the eligible pool before the training corpus was sampled. Role precedence is final test, then controller validation, then validation, then train, so a molecule appearing in two raw libraries is excluded from the lower-precedence role. The one such case is excluded from training and appears in neither the final training set nor the matched reserve.

Both evaluation panels are exactly disjoint and near-duplicate disjoint from the training source universe of 96,094 molecules. The matched reserve is cluster-disjoint by construction under an ECFP4 Tanimoto threshold of 0.95 with formula and scaffold blocking, and the external cohort shows zero exact overlap under a scaffold-matched comparison. The reserve’s higher scaffold sharing is by design, since it was carved from the same population, which is what makes it population-matched. The evaluation ladder and its no-rescue rules were recorded before the data. The validation panel is opened once, whatever it returns is banked, and no hyperparameter is re-chosen afterwards.

A pre-freeze mechanical smoke test executed one official benchmark source before the stop condition took effect. Its output was never inspected, is excluded from every analysis, and is retained only as an audit artifact, and we therefore describe the official cohort as inferentially unopened rather than literally never executed. The sealed final-test role, comprising 33,683 source records, has not been leakage-audited, that audit is a precondition for opening it, and no result in this paper draws on it.

D MODEL AND TRAINING DETAILS

Three learned objects appear in this paper and are trained separately, namely the goal-independent reference law R_θ , a property model $\hat{F}_\psi$ that defines molecular desirability, and goal- and budget-conditioned future-value models. Only the future-value models are trained against a downstream design objective, and R_θ never sees one.

D.1 REFERENCE EDIT LAW

R_θ is fitted once, frozen, and never retrained for any experiment reported here. Its identity is pinned by the sampling-law hash recorded in the corpus binding artifact and by the dataset, library, packed-store and reserve digests alongside it. The conditional edit law is normalized over the legal edit set as in (6) and is hierarchical as in (7), where a shared molecular encoder produces a state representation of width 256, a family gate scores which rewrite family fires, and family-specific scorers rank legal operands and payloads within it. Normalization is restricted to candidates the executor has admitted, so the model never places mass outside $\mathcal{A}(x)$. The encoder reads the current molecular graph and a progress variable, and does not read the source lead, the objective, the remaining budget, or any task constraint.

The reference law is trained by maximum likelihood of the next committed molecular successor under the declared editing training measure. For a productive teacher transition $x_i \rightarrow y_i^*$,

$$\mathcal{L}_{\text{ref}}(\theta) = -\frac{\sum_i w_i \log R_{\theta, t_i}(y_i^* | x_i)}{\sum_i w_i},$$

as in (8), where $R_{\theta, t}$ is the canonical successor kernel of Section 4.3 and terminal rows are excluded. For the frozen checkpoint used throughout this paper, teacher rates and importance weights are unit-valued, and the corpus sampler instead defines the reference measure through its registered family-weighted row distribution. The total event intensity is not optimized by this objective, and fixed-budget COMPOSE therefore does not claim calibrated continuous-time holding rates.

Training. Encoder depth **XXX**, message-passing steps **XXX**, optimizer and schedule **XXX**, batch size **XXX**, total optimization steps **XXX**, checkpoint-selection criterion **XXX**.

D.2 PROPERTY MODELS

Molecular desirability is defined through a frozen property model $\hat{F}_\psi$ rather than through oracle calls at control time. For the two-objective experiment it is fitted to a matched budget of 10,000 random-ZINC property labels, the same budget the comparator receives, and is shared identically by the immediate and future-aware arms so that any difference between them is attributable to control rather than to the landscape. Both arms therefore inherit the same prediction error and the same extrapolation behaviour outside the label range.

Architecture and training. Architecture **XXX**, featurization **XXX**, held-out predictive performance **XXX**.

D.3 QED FUTURE-VALUE MODEL

The region-goal controller estimates finite-budget reachability of a goal region and is used for the similarity-constrained editing experiments. The values below are design constants fixed in frozen preregistration documents before the corresponding data existed.

Inputs. The controller conditions on the region’s own coordinates, the signed margins of the current molecule to each of them, the remaining budget as a one-hot index, and a graph embedding of the current molecule together with the source lead and their difference and product, produced by the frozen reference-law encoder used unchanged with gradients never flowing into it. The feature width is $4 \times 256 + 2 + 2 + 2 + (24 + 1) = 1055$, comprising four embedding blocks, two state properties, two region thresholds, two signed margins, and a 25-slot budget one-hot. The budget ceiling of 24 is frozen because the released checkpoint was trained against a 25-slot one-hot, and a longer horizon is expressed by passing a larger ceiling to the feature builder rather than by editing the constant, which would change the input width and make the checkpoint unloadable.

Architecture and objective. The head is a multilayer perceptron $1055 \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 1$ with rectified-linear activations and dropout 0.1 after the first two hidden layers. It emits a logit, and the sigmoid is applied outside the module, so the multiplicative factor in (17) lies in $[0, 1]$ by construction. The loss is binary cross-entropy against the Monte-Carlo hitting label plus a Bellman consistency term at weight 0.3 on observed transitions, whose target is the sigmoid of the head at the successor. Optimization uses Adam at learning rate 10^{-3} with weight decay 10^{-5} for at most 40 epochs with early stopping at patience 5, and the best-validation checkpoint is selected rather than the last epoch.

Goal conditioning. Conditioning is continuous in the goal, so a threshold pair never seen in training is a point in the same input space rather than an unseen class label. The registered grid is the product of five QED thresholds $\{0.75, 0.80, 0.85, 0.90, 0.95\}$ and four similarity floors $\{0.30, 0.40, 0.50, 0.60\}$, giving 20 jointly trained goal regions, and the benchmark’s region $(0.90, 0.40)$ is one member and is given no special weight. The boundary condition $h_\phi \equiv 1$ on the region is enforced at inference, and in-region states are excluded from training rather than supervised toward it.

Splitting. The validation fraction is 0.15, held out by source molecule rather than by training example, because a trajectory rolled out of one lead yields many near-duplicate states and holding out a random fraction of examples would report memorization as generalization.

D.4 MULTI-OBJECTIVE FUTURE-VALUE MODEL

The multi-objective controller estimates a different quantity from the region model above and is documented separately for that reason. Its target is the expected preference-weighted terminal desirability under the frozen reference process, $h_b(x, \lambda) = \mathbb{E}_{R_\theta}[g_\lambda(X_b) \mid X_0 = x]$, with g_λ the Chebyshev desirability of (21), and it is a soft expectation rather than a hitting probability, so the binary construction of the region model does not carry over.

For numerical conditioning, the implementation evaluates the equivalent centered form $\tilde{g}_\lambda(x) = \exp(-(L_\lambda(x) - L_\lambda^*)/\tau_\lambda)$, where L_λ^* is a per-preference constant with respect to x . This multiplies all desirabilities for a fixed preference by the same positive constant, so the factor cancels in the normalization of (17) and leaves every controlled transition unchanged.

Supervision is Monte Carlo, with M independent continuations of the frozen reference law from a state giving the estimator of (16). The variance of that estimator grows with the remaining budget while the spread of true values across candidate successors shrinks, so target reliability degrades with horizon. Appendix F.4 reports that degradation and the rollout counts it implies.

Because supervision quality is horizon-dependent, learned future-value control is admitted only at horizons whose target reliability meets a criterion fixed in advance of any optimization result. The admitted horizon b_{FA} and the corresponding rollout count are **XXX**, and the deployed score is $h_\phi(y, \lambda, b - 1)$ for $b \leq b_{\text{FA}}$ and $g_\lambda(y)$ otherwise.

D.5 TRAINING-PIPELINE SCHEMATIC

[Schematic to be drawn. No data plotted.]

1. pairs $\rightarrow$ certified edit programs $\rightarrow$ successor-likelihood training $\rightarrow R_\theta$ (frozen)
2. frozen- R_θ rollouts $+ z \rightarrow$ future-value labels $+ \text{Bellman} \rightarrow h_\phi$
3. $\hat{F}_\psi$ read at the successor vs. $\hat{F}_\psi$ propagated through R_θ

Figure 4: **What is trained, and what each trained object is asked to know.** The reference edit law is fitted from certified programs of executable edits and frozen thereafter, reading neither the source lead nor the objective. The goal-conditioned value is fitted from rollouts of that frozen law, with the budget axis explicit and validation held out by source molecule. The third row is the immediate-versus-future-aware contrast. Nothing here is a result.

E SAMPLING AND CONTROL

Two realizations of the controlled law (17) are used. One is exact given the learned value. The other is an approximation whose error is in particle allocation rather than in the definition of success. Neither is a separate contribution.

E.1 EXACT FINITE-HORIZON CONTROL

Where the set of molecular states reachable within the remaining budget can be enumerated, the backward value of (12) is computed by exact dynamic programming. Initialize $h_0 = g_z$ on the enumerated terminal layer and sweep backwards, at each budget taking the expectation of the next layer’s value under R_θ restricted to the legal edit set. The controlled kernel of (13) follows by division and Theorem 3 applies without approximation. This is the only setting in which the word *exact* is used of a deployed computation.

E.2 MOLECULAR-SCALE SAMPLING

At scale h_ϕ replaces the exact value. Because it is a state-level function taking values in $[0, 1]$, the controlled row admits a rejection realization. Propose $Y \sim R_\theta(\cdot | x)$, execute, canonicalize, reject self-transitions, and accept with probability $h_\phi(Y, z, b - 1)$. Conditioned on acceptance the move has exactly the $R_\theta \cdot h_\phi$ law, and aliased successors aggregate correctly because the acceptance probability depends only on the resulting molecule. The proposal cap is finite, however, so when every proposal at a state is rejected the trajectory ends. The capped procedure is therefore not an exact realization of the controlled kernel and is not described as one. Cap pressure is recorded as a diagnostic with a preregistered escalation to the sequential realization below, which changes inference only.

When the desired region carries little mass under the reference process, control is realized as a twisted Feynman–Kac particle system. Every particle proposal comes from the frozen reference law alone, and h_ϕ enters only through the incremental weight (18). Using the value to both select and weight would double-count control and sample a different process. The weight product telescopes along a trajectory to $h_0(x_H)/h_H(x_0)$, and the terminal boundary $h_0(x) = \mathbf{1}[x \in \mathcal{B}_z]$ is exact, so a learned twist changes particle allocation without changing the terminal success criterion. The operating point is frozen and no particle-count sweep is permitted, with $N = 32$ particles, horizon 24, absorbing stop on entry to the goal region, and systematic resampling whenever the effective sample size $\text{ESS}(w) = (\sum_i w_i^2)^{-1}$ on normalized weights falls strictly below $N/2$. Resampling is systematic rather than multinomial, a single uniform draw positioning the whole comb. Per-transition values, per-step resampling decisions and resampling indices are recorded so the mechanical checks run offline against the artifact.

Candidate accounting follows one rule. One run returns one molecule. A source’s twenty candidates are twenty independent runs, each contributing the single molecule drawn from its terminal normalized particle measure. The particles within a run are inference state and are never counted as candidates, and intermediate states are never retrospectively harvested. A run reaching the budget without entering the region is recorded under a frozen extinction status rather than replaced. For a region goal the controller stops at the first committed molecule inside the region, an online stopping time evaluated on the realized state.

E.3 MULTI-OBJECTIVE CONTROL

Under a preference λ the immediate arm weights successors by $g_\lambda(y)$ and the future-aware arm by $h_\phi(y, \lambda, b - 1)$, with the reference law, legal edit set, property model, sources, preferences and budget identical between them. Where the admitted horizon b_{FA} is shorter than the optimization campaign, a campaign consists of repeated controlled trajectories of length at most b_{FA} launched from the frozen initialization bank and its archive until the common evaluation budget is exhausted. The campaign horizon and the horizon over which learned foresight is claimed are therefore distinct quantities.

Algorithm 1 Controlled molecular sampling, sequential realization

```
1: input: source  $x_0$ ; goal  $z$ ; budget  $K$ ; frozen  $R_\theta$  and  $h_\phi$ ; particles  $N$ ; optional rule-closed  
   predicate  $C$   
2: initialize  $x^i \leftarrow x_0$  and  $w^i \leftarrow 1/N$  for  $i = 1, \dots, N$ ;  $b \leftarrow K$   
3: while  $b > 0$  and some particle is unabsorbed do  
4:   for each unabsorbed particle  $i$  do  
5:     enumerate  $\mathcal{A}(x^i)$ ; if  $C$  is given, restrict to  $\mathcal{A}_C(x^i)$   
6:     push forward through the executor and canonicalize to  $\mathcal{S}(x^i)$   
7:     draw  $y^i \sim R_\theta(\cdot \mid x^i)$  {proposal from the reference law alone}  
8:      $w^i \leftarrow w^i \cdot h_\phi(y^i, z, b - 1) / h_\phi(x^i, z, b)$  {the value enters only here}  
9:      $x^i \leftarrow y^i$ ; if  $x^i \in \mathcal{B}_z$  then absorb particle  $i$   
10:  end for  
11:  normalize  $w$   
12:  if  $\text{ESS}(w) < N/2$  then  
13:    systematically resample the particles and reset  $w^i \leftarrow 1/N$   
14:  end if  
15:   $b \leftarrow b - 1$   
16: end while  
17: return one molecule drawn from the terminal normalized particle measure, or the frozen extinction  
    status if no particle was absorbed
```

E.4 RETARGETING AND PATHWISE CONSTRAINTS

Retargeting replaces the goal context at a realized state and continues with the remaining budget by re-evaluating (17) at that state, and no parameter of the reference process changes and the prefix is retained. A rule-closed constraint is imposed by filtering the legal edit set to $\mathcal{A}_C(x)$ of (22) before the transition is sampled, so no infeasible successor is proposed, weighted, or committed.

F EXPERIMENTAL DETAILS

Each subsection below gives the setup, arms and statistics for one experiment, together with the extended results omitted from the main text, in the order the experiments appear in Section 5.

F.1 REFERENCE-LAW EVALUATION

We evaluate the frozen reference law on held-out transitions from the matched reserve by measuring the probability assigned to the realized canonical successor. The uniform, empirical-family and learned laws are evaluated on the identical executable support and canonical successor quotient, and only their probability assignments differ. The empirical-family law uses corpus-level edit-family frequencies renormalized over the families available at the current state and distributes each family’s mass uniformly across its available successors. Two intermediate arms decompose the comparison, an empirical family with learned identity, which keeps corpus family frequencies and learns only which successor within the chosen family is realized, and a learned family with uniform identity, which learns only which family fires. Together they separate choosing which edit to make from choosing where to apply it.

The primary quantity is the mean negative log-probability of the realized successor, with the scored transition as the unit and paired differences by construction, since every law scores the same transition. Cells below a preregistered minimum size of 20 are not reported separately, and no scored transition was skipped and no partition failed to compile. Performance against the uniform law tests whether learning improves on executable support alone, and performance against the empirical-family law tests whether the reference law captures state-dependent preference beyond global family frequency. Neither comparison is interpreted in terms of physical molecular kinetics or synthetic-route likelihood.

The decomposition is lopsided. Learning only which family fires recovers little, improving on corpus family frequencies by 0.20 nats overall, whereas learning only where within a family the edit lands recovers most of the effect at 4.10, and the full model adds a further 0.20 beyond that. In the terms

Capability cell	<i>n</i>	unif.	emp. fam.	<i>emp+id</i>	<i>fam+unif</i>	<i>R_θ</i>
atom_restate: element identity change	725	6.21	6.54	4.84	7.28	5.58
atom_delete: leaf death	601	6.20	3.36	2.68	3.69	3.01
atom_insert: one-neighbor birth	572	6.14	7.12	5.23	6.97	5.09
bond_reroute: single-atom pendant, cyclic	464	6.24	5.26	4.07	5.48	4.29
bond_reroute: multi-atom pendant, cyclic	216	6.32	5.56	4.38	6.14	4.96
ring_system_restate: dearomatization [†]	202	6.25	2.84	2.97	1.72	1.85
ring_system_restate: aromatization [†]	165	6.51	2.85	2.34	1.12	0.61
cycle_insert: close to monocyclic [†]	138	6.47	7.19	3.27	4.80	0.88
cycle_attach: open from monocyclic [†]	104	6.16	5.24	5.20	3.34	3.29
bond_reorder: bond-order decrease	83	6.32	4.30	4.01	3.18	2.89
bond_reorder: bond-order increase	83	6.49	4.71	4.40	3.29	2.99
cycle_insert: close to polycyclic [†]	81	6.55	7.37	3.95	5.26	1.84
cycle_attach: open from polycyclic [†]	69	6.42	5.82	5.71	4.08	3.96
Overall	3545	6.25	5.37	4.10	5.17	3.90

Table 5: Negative log-probability of the realized canonical successor, by capability cell, lower is better. All five laws are evaluated on the same executable support and the same canonical quotient, and only the probability assignment differs. Columns *emp+id* and *fam+unif* are the two decomposition arms defined above, empirical family with learned identity, and learned family with uniform identity. Cells below the preregistered minimum size of 20 are not reported separately. No scored transition was skipped and no partition failed to compile. [†]Supervision for these cells comes entirely from the synthetic-perturbation lane. See the provenance note below.

used in Section 5.1, the empirical-family-to-full improvement is 1.4616 nats and the identity-only arm recovers 1.2674 of it, or 86.7%, against 13.3% for the family gate alone. The reference law’s advantage is therefore largely an advantage in placing an edit on a particular molecular site rather than in selecting the operator family. The empirical-family control is also not uniformly strong. On both *cycle_insert* cells, on *atom_insert* and on *atom_restate* it is worse than assigning equal probability to every distinct successor, so on birth and insertion families a corpus frequency prior misallocates mass relative to ignorance. On four cells the empirical-family/learned-identity ablation assigns higher probability to the realized successor than the full model, indicating that the learned family gate does not uniformly improve over corpus-level family frequencies and that the identity model carries the result on those families.

Aggregating by supervision provenance sharpens this. On the 2,744 transitions from families with data-backed supervision, the reference law retains a 1.06-nat advantage over empirical family frequencies (5.55 against 4.49), so the main-text claim survives the obvious confound. On that subset, however, the empirical-family/learned-identity ablation reaches 4.25 nats and therefore assigns higher probability to the realized successor than the full model does, and the learned-family/uniform-identity arm at 5.79 is worse than corpus family frequency. The learned family gate is thus not uniformly beneficial, and on data-backed families the identity model carries the result in spite of it. The complementary synthetic-only subset comprises 759 transitions, where the reference law reaches 1.79 nats against 4.72 for empirical family frequencies, and the aggregate improvement reported in Section 5.1 is correspondingly larger than the data-backed figure. The partition is computed from the emitted family-lane provenance artifact rather than from a hand-specified family list, and the restricted aggregates are reproduced from it.

The cells in which the reference law performs best are those whose supervision is synthetic, as recorded in Appendix C. On the ring and cycle families the model recovers a perturbation of a kind it was trained to invert, and the aggregate improvement should not be read as evidence about molecular kinetics. Resolving instead by the number of available successors gives 4.86 nats for states with no alternative in the recorded band and 5.51, 5.44 and 5.51 for successor counts of 1–4, 5–24 and 25 or more under the empirical-family law, against 4.34, 4.03, 4.00 and 4.21 for the learned-identity arm, so the learned advantage is not an artifact of nearly deterministic rows.

F.2 FUTURE-REACHABILITY EVALUATION

The panel was sealed under preregistration and opened once. Sixty-seven held-out source–target pairs were constructed whose target is reachable under the frozen executable process within the declared budget. Two were excluded from the primary analysis under a committed amendment, giving 65 pairs, and the full 67 are retained as a sensitivity analysis. All arms share the reference

law, the legal edit sets, the target molecule and the edit budget. A myopic policy selects the locally preferred edit, whereas verified lookahead evaluates candidate successors by whether the target remains recoverable within the remaining budget, and three intermediate prioritizers restrict which successors are expanded before lookahead is applied. Continuation work is reported as a fraction of the candidates evaluated by all-candidate lookahead, so recovery and cost are read together. The unit is the source–target pair and intervals are paired bootstrap.

Arm	Rec.	Rate	Cont./pair	Frac. univ.	Gain ret.
Greedy local decision	26/65	40.0%	0.0	0%	—
R_θ top-1 + lookahead	33/65	50.8%	7.3	34.7%	0.50
Similarity top-1 + lookahead	34/65	52.3%	7.3	34.7%	0.57
Similarity top-2 + lookahead	37/65	56.9%	10.8	52.1%	0.79
Goal-aware h_ϕ top-1 + lookahead	40/65	61.5%	7.2	34.6%	1.00
All-candidate lookahead	40/65	61.5%	20.5	100%	1.00

Table 6: Sealed panel, 65 pairs. *Rec.*, targets recovered. *Cont./pair*, candidate continuations actually evaluated per pair. *Frac. univ.*, that count as a share of what all-candidate lookahead evaluates. *Gain ret.*, the fraction of all-candidate lookahead’s 14 rescues the arm also recovers.

Of the 65 pairs, 14 are recovered by all-candidate lookahead and not by greedy, and none by greedy and not by all-candidate lookahead. That one-sidedness is a property of the decision rule rather than a finding, since the planner overrides greedy only on strict improvement in remaining-budget value and ties keep the greedy action, so an arm cannot lose a target greedy would have reached, and across the panel 215 tied candidates kept the greedy action and no arm regresses against greedy. A discordance test would therefore report a foregone conclusion, and the reported quantities are instead the paired difference in recovery rate, 21.5 percentage points [12.3, 33.5], and the share of greedy’s 39 failures that lookahead rescues, 35.9% [21.2, 52.8].

Equal recovery counts do not imply equal rescue sets. The goal-aware prioritizer and all-candidate lookahead both recover 40 targets but not the same 40, since the prioritizer rescues one pair all-candidate lookahead misses and misses one pair it rescues, overlapping on 13 of 14. The similarity and reference-probability arms are strictly nested inside all-candidate lookahead’s rescue set, missing six, three and seven of it respectively. Resolving by verified path length, over pairs reachable in four edits ($n = 21$) the arms recover 7, 12 and 13 for greedy, all-candidate lookahead and the goal-aware prioritizer, at five edits ($n = 34$) they recover 14, 20 and 19, and at six edits ($n = 10$) they recover 5, 8 and 8. The original hypothesis that the advantage would widen with path length was tested and not supported, and horizon is reported as an observation. Including both excluded pairs, greedy recovers 26/67 and all-candidate lookahead 41/67, a headroom of 15 rather than 14, and no ordering changes. The panel consumed 1,347 kernel calls.

F.3 SOURCE-CONDITIONED MOLECULAR EDITING

The protocol is adopted from the comparator and is given in Appendix G.1. Each COMPOSE candidate is the output of one controlled trajectory terminating prospectively at the first realized state inside the goal region, and intermediate molecules are not retrospectively collected. The reference law is unchanged from Appendix F.1, and the future-value controller is trained and frozen on data disjoint from the official sources before the cohort is opened. A size-fixed internal ablation removes every successor that changes the current heavy-atom count while holding controller, reference law, sources, seeds and candidate allowance fixed. The unit is the source molecule, and the candidate curve is descriptive and carries no significance claim, and it is drawn only over the k actually measured, which is $k = 1, \dots, 12$ on the prospective panel and $k = 1, \dots, 20$ once the official cohort is opened. The official cohort is opened once and whatever it returns is banked, including a disappointing result, with no hyperparameter re-chosen afterwards.

A receding one-step terminal tilt of the frozen reference law by the immediate property value of each successor was preregistered as the inference policy and refuted on the disjoint development panel, with 0 successes in 320 trajectories, before any official-cohort outcome existed. It is retained as a developmental negative control. What this licenses is narrow. The finite-horizon controller is a pre-outcome amendment consistent with the paper’s construction, motivated by a developmental

failure of a cheap one-step approximation. It is not a restoration of an originally frozen policy, and compute cost was not the only consideration separating the two.

The official-cohort success rate, its candidate curve and the size-fixed ablation are **XXX**.

F.4 MULTI-OBJECTIVE MOLECULAR OPTIMIZATION

Preference-conditioned optimization over JNK3 and GSK3 β follows the comparator’s setting, detailed in Appendix G.3. A common objective-blind initialization bank of 100 molecules is fixed before either method is evaluated, and both receive a matched supervision budget of 10,000 property labels, the same five preference vectors and the same 5,000-evaluation optimization budget. Average Pareto score is primary, with hypervolume, novelty and top- K diversity guarding against scalar gains obtained by collapsing onto a narrow molecule set. The unit is the run, replicated over the frozen bank. Only the immediate-versus-future-aware contrast isolates the contribution of molecular-process control, because COMPOSE and the comparator do not share a property predictor.

Before training a multi-objective future-value model, we measured how well its Monte Carlo target can be estimated. On 24 held-out states with 16 candidate successors each and 32 reference continuations per candidate, scored by the frozen property model with no oracle calls, the rank correlation between immediate desirability and true future value has per-state median 0.518 [0.435, 0.700] at four remaining edits and 0.318 [0.181, 0.421] at eight, with intervals resampled over states. The two criteria select different successors on 68.3% [55.8, 80.8] and 84.2% [73.3, 93.3] of state–preference decisions respectively, and following future value rather than immediate desirability improves realized continuation value by a median 12.2% [1.6, 19.8] at four edits and 19.0% [13.1, 25.9] at eight.

Target reliability itself degrades with horizon. Estimated by split-half agreement between disjoint sets of continuations, a 32-rollout target has reliability approximately 0.66 at four remaining edits and 0.39 at eight, implying by the Spearman–Brown relation that roughly 65 and 200 continuations respectively would be required to reach a reliability of 0.80. Because supervision quality is horizon-dependent while the measured benefit is not, the supported horizon is fixed from target reliability and computational feasibility rather than from optimization performance, as recorded in Appendix D.

Table 7: Two-objective molecular optimization at a matched oracle-label budget. The estimand is the average score of the top-100 molecules (APS) with novelty and top- K diversity, and the independent statistical unit is the run, replicated over the frozen initialization bank. The comparator’s published row is *reported context* under an unspecified initialization and an unstated hypervolume reference point, and is never the quantity a COMPOSE number is described as beating, and the matched rerun is the row that adjudicates. Every COMPOSE cell and every matched-rerun cell is unmeasured.

Arm	Initialization	APS	Novelty	Diversity
Comparator, as published (<i>reported context</i>)	unspecified	0.841	100%	0.768
Comparator, matched rerun	frozen common bank	XXX	XXX	XXX
COMPOSE, future-aware (17)	frozen common bank	XXX	XXX	XXX
COMPOSE, immediate	frozen common bank	XXX	XXX	XXX
Four-objective escalation	frozen common bank	XXX	XXX	XXX

F.5 MID-TRAJECTORY RETARGETING

We evaluate retargeting on 40 source-disjoint held-out molecules under potency-first and developability-first histories. Each prefix contains three committed edits within a six-edit horizon and is fixed before the eventual joint objective B is constructed, so every post-switch comparison begins from an identical realized prefix with the same remaining budget. Margins are normalized by the held-in interquartile range and the reported objective is the worst margin of the conjunction, with zero denoting satisfaction. At the switch the median worst margin is -1.115 following potency-first prefixes and -1.810 following developability-first prefixes, so neither history already satisfies B .

Retargeting materially redirects both histories, and under the primary verified comparison continuing from the realized state remains preferable to restarting from the source. The mean advantage is $+0.302$ [0.186, 0.413] following potency-first prefixes and $+0.176$ [0.069, 0.283] following developability-first prefixes, with wins on 35/40 and 29/40 sources respectively. The clairvoyant controller remains

Comparison	History	Mean	Median	95% CI	W/L
Responsiveness	potency	+0.748	+0.794	[+0.608, +0.891]	36/0
	developability	+1.462	+1.357	[+1.240, +1.691]	40/0
Greedy-matched history	potency	+0.367	+0.338	[+0.251, +0.482]	37/3
	developability	+0.211	+0.147	[+0.065, +0.361]	26/14
<i>Value of history</i> (primary)	potency	+0.302	+0.243	[+0.186, +0.413]	35/5
	developability	+0.176	+0.127	[+0.069, +0.283]	29/11
Price of surprise	potency	-0.252	-0.123	[-0.364, -0.155]	6/32
	developability	-0.428	-0.291	[-0.580, -0.288]	4/36

Table 8: Held-out retargeting panel, 40 source-disjoint sources per history. Rows compare, in order, retargeting against continuing the old goal, greedy retargeting against greedy restart from the source, verified continuation from x_3 against verified restart from x_0 (the primary estimand), and verified continuation against a clairvoyant controller given B from the first edit. Entries are paired post-switch differences in the registered objective, and positive values favor the first strategy named. W/L counts sources on which that strategy is better or worse, with ties omitted. Intervals are source-level bootstrap.

better, giving the negative price-of-surprise effects shown in the table. Arm-level medians follow the same ordering under both histories. Continuation from the realized molecule lies above restart and below clairvoyance, at -0.016 and $+0.077$ for greedy and verified retargeting against -0.532 and -0.428 for the two restart arms and $+0.389$ for clairvoyance under potency-first prefixes, and correspondingly under developability-first prefixes.

The effect decreases from development to held-out confirmation, with the primary differences shrinking from $+0.444$ to $+0.302$ and from $+0.290$ to $+0.176$. The confirmation panel contains genuine losses in both histories rather than a deterministic ordering, which is the property that distinguishes the effect from a structural guarantee. We do not use verified-versus-greedy policy improvement as an estimand because its direction follows from the decision rule itself. Prefix revisiting is negligible, occurring for at most one source per arm and history, so the effect is not an artifact of trajectories folding back onto their own prefix.

F.6 PATHWISE CONSTRAINTS

The pathwise experiment remains developmental. The cLogP corridor was selected following an earlier feasibility screen, and a separately frozen held-out confirmation has therefore been specified but not opened. Endpoint-only control constrains only the returned molecule. Pathwise control removes any successor outside the corridor before the transition is sampled. A terminal-penalty arm changes the objective but leaves the transition support unchanged. All arms share the reference law, objective, horizon, sources, seeds and evaluation budget, and the unit is the source molecule with source-clustered intervals.

On 24 eligible sources, endpoint-only trajectories return to the corridor after leaving it in $16/24 = 0.667$ $[0.458, 0.833]$ cases under greedy control and $14/24 = 0.583$ $[0.375, 0.792]$ under verified control, and the conditional fractions are $16/24$ and $14/23$, so the small-denominator pathology the unconditional estimand guards against did not arise. These excursions are not confined to the boundary. Their median depth is 0.69 cLogP units and their median duration is four of the six committed edits. An earlier protected-motif experiment produced zero such excursions, confirming that the estimand need not be positive simply because intermediate states are observed.

Restricting the support to the corridor remains viable on this panel. The median source retains 79.8% of its legal successors, only one source reaches the prespecified support-tight threshold, no evaluated state has an empty constrained edit set, and one trajectory terminates. Excluding the support-tight source leaves prevalence at 0.652 and 0.565 and terminal costs at -0.121 and -0.041 , so the conclusions are unchanged.

The developmental panel does not resolve the terminal-utility cost of imposing the constraint. The paired change in normalized DRD2 utility is -0.105 $[-0.377, 0.099]$ under greedy parity, with 5 wins, 8 losses and 11 ties, and -0.023 $[-0.182, 0.134]$ under verified parity, with 9 wins, 10 losses and 5 ties. Both medians are exactly zero and both intervals span zero, which does not establish equivalence, since the greedy interval still permits a loss of 0.377 held-in interquartile-range units.

Zero violations under support restriction is an integrity consequence of masking rather than an empirical performance statistic, and the apparent advantage of future-aware control under the mask, $+0.661$ $[0.428, 0.944]$, is a ceiling rather than a null, since greedy pathwise control completed all 24 sources and left no binary headroom.

Constraint handling	Complete-path feasible	Wasted path-infeasible evaluations	Final DRD2 utility
Endpoint rejection	XXX	XXX	XXX
Terminal penalty	XXX	XXX	XXX
Support restriction	100% by construction	0% path-infeasible	XXX

Table 9: Three-arm comparison for Section 5.6. The 100% complete-path feasibility and 0% wasted path-infeasible evaluations of the support-restriction arm are consequences of the construction and are *not* empirical superiority statistics. Successors leaving the corridor are absent from the controlled kernel, so no such state can be committed. The informative comparisons are the first two rows and the final-utility column.

The held-out confirmation fixes the corridor at $C = [2.3689, 4.4522]$ with the same six-edit horizon and endpoint-only semantics, at $n = 48$ sources. It passes only if both criteria clear at full α under intersection-union. The lower bound of the source-clustered interval on hidden-path prevalence must exceed $1/3$, a threshold fixed before the development run, and the lower bound on the mean paired change in terminal utility must exceed -0.25 held-in interquartile-range units, with -0.20 as a prespecified sensitivity analysis. The panel is sized on the joint criterion, giving power 0.976 at the 0.25 margin and 0.874 at 0.20. No held-out source has been opened.

A value head retrained to be aware of a longer horizon was built, evaluated and rejected on four separate measurements, and is not the head used anywhere in Section 5. The consequence for the main text is a claim boundary. Any gain from a longer horizon comes from giving the frozen process more edit opportunities, not from a better budget-conditioned value model.

G BENCHMARK AND BASELINE DETAILS

G.1 QED EDITING BENCHMARK

The protocol is adopted verbatim from the comparator, namely the 800 test molecules with $\text{QED} \in [0.7, 0.8]$ as sources, 20 candidates per source, a Tanimoto similarity constraint of 0.4 to the source, and success defined as at least one returned candidate reaching $\text{QED} \in [0.9, 1.0]$ while satisfying that constraint. Source selection, fingerprints, property calculations, candidate count and aggregation must be identical for the comparison to stand, and where any of these could not be matched it is stated rather than assumed.

Output contract. Each source receives exactly K candidate slots. A trajectory that reaches the target region returns the first committed molecule satisfying the benchmark criterion. If no sampled continuation reaches the region, the slot returns the unchanged canonical source solely to preserve the fixed- K benchmark interface. This output necessarily fails because benchmark sources begin below the target QED threshold. No failed slot is resampled or replaced. This contract is frozen before the official cohort is opened and is not revised afterwards.

Every non-source returned molecule is a benchmark success. All 371 non-extinct trajectories return a molecule in the target region, while all 1,165 extinct slots return the unchanged source and are guaranteed failures. A smaller candidate allowance can disadvantage both success and the diversity metric, because a source with only one successful distinct return contributes zero diversity. Separately, no returned success is identical to its source, so the count is unchanged under the stricter rule of the released success script, which requires similarity strictly below one.

Prior-method values are taken as reported in the comparator’s paper and are not re-derived. The latent-variable and reinforcement-learning rows arrive as inherited benchmark context rather than as chosen comparators, and no claim in this paper turns on any of them individually, and the insertion/deletion graph-diffusion result at 45.1% is the modern row that matters. Higher rates have been reported on this task outside the comparator’s table, and Appendix G.2 records them so that Table 1 is not read as the complete published record.

Diversity is computed with the released evaluation script of Jin et al. (2019) rather than reimplemented from its prose description, and we audited that script rather than infer its behaviour. It admits a candidate only if it satisfies both the similarity constraint and the property target, deduplicates the admitted molecules, excludes a source with no admitted molecule from the average, and assigns a diversity of exactly zero to a source with only one. Similarity is a Morgan fingerprint of radius two on 2048 bits with chirality disabled. These choices are not cosmetic. On our panel 19 of the 70 solved sources return a single distinct molecule and therefore enter the average as zero, which is the whole difference between the reported 0.393 and the 0.539 that averaging over only the 51 multi-molecule sources would give. The reported value is 0.393 with a bootstrap interval of [0.331, 0.452] over sources. Among solved sources COMPOSE returns on average 4.34 distinct successful molecules of the twelve permitted, and no successful candidate is identical to its source, so the success count is unchanged under the stricter rule of the released script, which requires similarity strictly below one. We recomputed drug-likeness and similarity from the returned structures with those same functions rather than trusting the values recorded during sampling, and the two agree to machine precision and the admission decision agrees on all 1,536 candidates. All returned strings are canonical, so the string-level deduplication of the released script coincides with deduplication by molecule, and no pair of distinct returned molecules shares a fingerprint under the achiral setting.

One limitation belongs to the comparator rather than to us. Ninniri et al. (2025) do not state how the diversity column of their main property-optimization table is computed. Their baseline diversity values are identical to those reported in the graph-to-graph translation line, which places the inherited rows on the scale defined above, but the method’s own entry is undocumented, and a rule matching the one used here is stated only in their appendix, for a separately adapted protocol with a different number of optimization rounds. We therefore report our value as computed under a named and audited definition, and we do not assert that it and the published entry were produced by the same procedure.

Because every COMPOSE trajectory terminates prospectively on entering the goal region, a run either returns a molecule satisfying both constraints or goes extinct, and the set the diversity script admits is exactly the set of successful returns. The admitted set is therefore constructed the same way as in the published rows rather than merely being similar to it, and the residual differences are the cohort of 128 against 800 and the allowance of twelve candidates against twenty. The allowance difference is not neutral for diversity, since a smaller allowance yields fewer second successes per source and so more single-molecule sources contributing zero, which depresses the reported value relative to what the same system would produce at twenty candidates.

The benchmark constrains returned molecules and says nothing about internal successor evaluations, and competing methods use their native inference procedures. The comparison therefore measures task performance under a fixed output allowance, no efficiency claim is made, and inference ledgers are reported separately. The entry currently printed is the 128-source prospective validation panel at twelve of the twenty permitted candidates. It is not the official cohort, it uses fewer outputs than the benchmark allows, and it is not presented as a matched row.

G.2 PUBLISHED RECORD ON THE QED EDITING TASK

Table 1 reproduces the comparator set assembled by GrIDDD. Stronger values have been reported on the same task by the graph-to-graph translation line, and we record them in Table 10 rather than leave the main table to be read as the full published record. We checked the primary sources rather than inheriting the numbers second-hand. The translation results use the same 800 source molecules, the same similarity threshold $\delta = 0.4$, and the same allowance of $K = 20$ decoded candidates per source (Jin et al., 2019). The rate frequently quoted as 60.6% is VJTNN+GAN, whereas VJTNN itself is reported at 59.9%. Jin et al. (2020) evaluate on the test sets provided by Jin et al. (2019) under the same range and similarity constraint with $K = 20$, and reproduce the 59.9% row, which is what places their 76.9% on the same cohort.

G.3 TWO-OBJECTIVE BENCHMARK

The comparator reports an average Pareto score of 0.841, novelty of 100%, diversity of 0.768 and hypervolume of 0.763 ± 0.031 . Three quantities needed to reproduce those numbers are not recoverable from the paper. The initialization is unspecified. An initialization per weight vector is

Method	Reported by	K	QED success ($\uparrow$)	Diversity ($\uparrow$)
MMPA	Jin et al. (2019)	20	32.9%	0.236
VSeq2Seq	Jin et al. (2019)	20	58.5%	0.331
VJTNN	Jin et al. (2019)	20	59.9%	0.373
VJTNN+GAN	Jin et al. (2019)	20	60.6%	0.376
AtomG2G	Jin et al. (2020)	20	73.6%	0.421
HierG2G	Jin et al. (2020)	20	76.9%	0.477

Table 10: **Further published results on the same similarity-constrained QED editing task.** All rows use the 800-source cohort of Jin et al. (2019) with $\delta = 0.4$ and $K = 20$. These are supervised graph-to-graph translation models trained on parallel source-target pairs curated for the target property, whereas the COMPOSE reference law is trained once without reference to QED and the property enters only through control. The rows therefore measure different problem settings and we do not present them as an ablation of one another, nor is any COMPOSE claim stated relative to them.

referred to, but no bank, distribution or seed is given, and which molecules a preference-conditioned optimizer starts from changes the difficulty of the benchmark. The hypervolume reference point for the molecular tasks is never stated, so a hypervolume computed by us is not on the comparator’s scale. The released implementation appears to evaluate a single preference vector, against the five described in the paper.

Rather than guess at the missing initialization, we freeze a common, deterministic, objective-blind bank before either method is evaluated and run the comparator ourselves from that bank under a matched label budget, the same five preferences and the same optimization budget. The comparator’s published values are reported as context under a different and under-specified initialization, and the matched rerun is the row that adjudicates, and the published figure is never the quantity a COMPOSE number is described as beating.

Average Pareto score, novelty and top- K diversity are primary because they are computed identically from a returned molecule set and require no shared reference geometry. Hypervolume is secondary under a preregistered origin reference point and is not comparable to the published value, whose reference point is unknown, and that non-comparability is stated wherever the number appears. Within COMPOSE, the immediate and future-aware arms share the reference process, legal edit sets, property model, preferences, bank and budget, which is what makes their difference attributable to control. Across methods, no property predictor and no compute accounting is shared, and no cross-method difference is attributed to the search procedure alone.

H SOFTWARE, RANDOMIZATION, AND COMPUTE

H.1 SOFTWARE AND ENVIRONMENT

Every job runs in a containerized Debian image pinned to Python 3.11 with `torch` 2.4.0, `numpy` 1.26.4, `scipy` 1.13.1 and `rdkit` 2024.3.5. Chemistry is pinned to a single RDKit release because canonical molecular identity, ring perception and sanitization all come from it and all three enter the definition of the state space in Appendix A.

Only artifacts satisfying the evidence criteria used in the manuscript are cited for scientific claims. Developmental artifacts lacking complete producer or input metadata are retained for audit purposes but excluded from claim-bearing results. Bulk artifacts such as corpora, embeddings, per-run records and trained heads live on a persistent volume rather than in the source repository, while result summaries are versioned alongside the code, so a fresh checkout can re-run every job but cannot read a banked result without the volume. The reproducibility package records the code revision, input identities and result summaries required to regenerate each reported analysis.

H.2 RANDOMIZATION AND STATISTICS

Run seeds are derived rather than drawn, by hashing a protocol tag together with the source molecule and the replicate index and taking the leading eight bytes of the digest. In paired comparisons the arm name is deliberately excluded from the seed, so arms sharing a source and replicate index

consume identical randomness and a difference between them cannot be a seeding artifact. Where an arm-specific seed is unavoidable for later replicates, the first replicate is still constructed to be identical across arms.

Intervals are bootstrap intervals resampled over the independent unit named in each experiment. That unit is the source molecule in most cases, the source–target pair in the sealed lookahead panel, and the state in the reliability analysis of Appendix F.4. Replicate counts differ by analysis, ranging from 4,000 to 8,192 draws, and each artifact records its own. Where an interval is exact rather than bootstrapped, that is stated with the reason.

H.3 COMPUTE

All reported jobs are CPU-only, and no result in this paper required a GPU. Sampling containers reserve one core and burst to eight under a six-hour ceiling, and driver containers hold a quarter core under a twelve-hour ceiling. Compute is reported in two units that are never pooled. These are kernel calls, meaning evaluations of the learned reference law, and container-seconds. The sealed lookahead panel consumed 1,347 kernel calls and roughly 24,300 container-seconds, and retargeting branch points cost on the order of 10^2 kernel calls and 10^3 container-seconds each. Oracle calls are counted separately again, because the benchmark budgets in Appendix G constrain returned molecules rather than internal evaluations.